

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech Computer Science and Engineering & Biosciences
Course Code & Course Name	CSA1202 - Computer Architecture
Academic Year / Batch	2025-2029
Faculty Name	Dr. J. Vasavi
Assignment Title	Precision Agriculture using Computer Architecture
Date of Issue	
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2 and CO3
Bloom's Taxonomy Level	L3 - Apply; L4 - Analyse
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger
Industry / Societal Relevance	Precision irrigation, water conservation and energy efficiency

B. Assignment Problem / Challenge

Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

- Apply course concepts rather than reproduce theory.
- Identify and analyze the problem.
- Consider requirements and constraints.
- Develop or compare alternative solutions where appropriate.
- Use relevant modern engineering/computing/design tools.

- Interpret results.
- Make and justify engineering decisions.
- Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

A smart irrigation system uses soil-moisture, temperature, and humidity sensors to determine the required amount of water for crops. Design a pipelined processor datapath to process sensor values continuously. Identify data, structural, and control hazards; apply forwarding, stalling, and branch prediction; and evaluate how pipeline optimization affects computation time and water/energy consumption.

D. Requirements and Constraints

Students shall identify or be provided with relevant requirements and constraints.

Examples include:

- Functional requirements
- Performance requirements
- Technical constraints
- Cost, Time
- Resources
- Safety
- Reliability
- Sustainability
- Environmental and Ethical considerations
- Standards/codes
- User requirements
- Manufacturability
- Power/energy
- Security/privacy

Only constraints relevant to the particular course/problem need to be considered.

E. Student Work

1. Problem Understanding and Formulation

Explain:

- What is the problem?
- What are the expected outcomes?
- What information/data is available?
- What assumptions are made?

- What constraints must be satisfied?

2. Application of Course Knowledge

Identify and apply the relevant:

- Principles
- Theories
- Mathematical models
- Algorithms
- Engineering concepts
- Scientific concepts
- Design methodologies

Show necessary calculations, derivations or reasoning.

3. Solution / Design / Methodology

Develop the proposed solution. Depending upon the nature of the course, this may include:

- Design
- Algorithm
- Model
- Circuit
- Architecture
- Experiment
- Process
- Prototype
- Simulation
- Case analysis
- Data analysis
- Mathematical solution

Where appropriate, consider **more than one possible solution**.

4. Use of Modern Tools

Use appropriate tools wherever relevant.

Examples:

CSE/IT: Python, MATLAB, TensorFlow, cloud platforms, Git, simulation tools

ECE/EEE: MATLAB/Simulink, Cadence, Synopsys, Vivado, Multisim, LTspice

Mechanical: ANSYS, SolidWorks, CATIA, MATLAB

Civil: STAAD.Pro, ETABS, AutoCAD, Revit

Biomedical: MATLAB, LabVIEW, signal/image-processing tools

Biotechnology: Bioinformatics/data-analysis/simulation tools

Students shall provide appropriate evidence of tool usage and outputs.

5. Results and Validation

Present the results using appropriate:

- Tables
- Graphs
- Simulation outputs
- Experimental observations
- Screenshots
- Performance metrics
- Statistical analysis
- Test results

Validate whether the proposed solution satisfies the stated requirements.

6. Analysis and Engineering Decision

Interpret the results.

Where applicable:

- Compare alternatives.
- Identify advantages and limitations.
- Discuss trade-offs.
- Explain why the final solution was selected.
- Justify the decision using quantitative or qualitative evidence.

7. Broader Considerations

Where relevant to the problem, discuss the impact of the proposed solution with respect to:

- Sustainability
- Environment
- Society
- Safety
- Ethics
- Economics
- Accessibility

- Professional responsibility

8. Conclusion

Summarize:

- Proposed solution
- Major findings
- Achievement of requirements
- Limitations
- Possible improvements

9. Student Reflection

- What did you learn from this assignment beyond what was directly taught in the classroom?
- If you were given additional time/resources, what would you improve and why?

10. References

Use an appropriate citation format and acknowledge all external sources, datasets, standards, software used.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

B. Assignment Problem / Challenge

Problem Statement

A smart irrigation system uses soil-moisture, temperature, and humidity sensors to determine the required amount of water for crops. Design a pipelined processor datapath to process sensor values continuously. Identify data, structural, and control hazards in the processing sequence and apply suitable techniques such as forwarding, stalling, and branch prediction. Evaluate how pipeline optimization can reduce computation time and water/energy consumption.

C. Problem Statement

The assignment is implemented as a solar-powered, rainwater-aware greenhouse irrigation controller executed by a cycle-accurate five-stage processor simulator. Sensor values are converted to Q8.8 fixed-point words, processed by a load/store RISC instruction stream, and used to compute moisture deficit, an evapotranspiration correction, valve time and water volume. Booth radix-2 multiplication and non-restoring division execute as multi-cycle EX-stage operations. The same control program is then run on six processor configurations ranging from a sequential baseline to an optimized pipeline with forwarding, split instruction/data memories and a 16-entry two-bit branch predictor. Functional correctness, hazard behavior, cycle count, CPI, throughput, branch accuracy, energy and irrigation decisions are measured from executable results.

D. Requirements and Constraints

Requirement / Constraint	Applied in this implementation
Functional	Read moisture, temperature and humidity; reject invalid readings; decide irrigation; enforce battery/tank safety; store valve time and litres.
Processor	Use a IF-ID-EX-MEM-WB datapath with explicit latches, clock-cycle advancement, register file, memory and branch resolution.
Arithmetic	Use Q8.8 sensor arithmetic; implement Booth multiplication, restoring division, non-restoring division and IEEE 754 binary32 demonstration.
Hazards	Model RAW dependencies, load-use stalls, unified-memory structural conflicts, multi-cycle EX occupancy and control hazards.
Optimization	Compare sequential, basic pipeline, forwarding, split I/D memory, static not-taken and dynamic two-bit prediction.
Safety/ Reliability	Invalid sensor data, low battery and unavailable rainwater must not cause unsafe irrigation.

Requirement / Constraint	Applied in this implementation
Validation	Use deterministic unit/integration/system tests and compare processor outputs with an independent software reference model.
Resource/Cost	Software simulation only; no special hardware or paid software is required.
Input data	Controlled local sensor test cases and scenario inputs are used. No external dataset is required.

E. Student Work

1. Problem Understanding and Formulation

What is the problem?

A fixed-time irrigation controller waters every zone for the same duration even when the soil is already wet, when a sensor is faulty, when the rainwater tank is nearly empty or when the solar battery cannot run the pump. Replacing this timer with sensor-driven control improves resource use, but it also creates a realistic embedded computing workload containing arithmetic, memory accesses and conditional branches. The computer-architecture problem is to execute that workload correctly while reducing processor cycles and energy without changing the irrigation decision (Monteiro, Santos, and Gonçalves 2021; Hamouda et al. 2024).

What are the expected outcomes?

- A correct sensor-driven irrigation decision for valid, boundary and faulty sensor inputs.
- Correct Q8.8 and IEEE 754 representations, Booth multiplication and restoring/non-restoring division behavior.
- A cycle-accurate five-stage pipeline with data, structural and control hazard handling.
- Quantitative comparison of six execution configurations using cycles, CPI, IPC, throughput, latency, stalls, prediction accuracy and processor energy.
- Identical functional outputs across all processor configurations and agreement with an independent reference model.
- A browser-based demonstration that visualizes simulator outputs without re-implementing processor logic in JavaScript.

What information/data is available?

The project uses controlled simulation inputs rather than an external dataset. The file `data/sensor_tests.json` contains 13 deliberately selected sensor cases covering normal, threshold, environmental-stress and invalid-input conditions. The file `data/scenarios.json` defines three shared-

resource situations for the rainwater tank and solar battery. These inputs are deterministic so the exact arithmetic and hazard behavior can be reproduced.

ID	Case	Category	Moisture %	Temp C	Humidity %	Fault?
1	dry_loam_zone_a	dry soil	18.0	28.0	55.0	No
2	very_dry_hot_dry_air	dry soil + high temperature + low humidity	8.5	38.0	22.0	No
3	saturated_after_rain	wet soil	62.0	24.0	78.0	No
4	exact_target_threshold	threshold condition	45.0	25.0	60.0	No
5	one_step_below_target	threshold condition	44.75	25.0	60.0	No
6	heat_wave_midday	high temperature	30.0	44.0	40.0	No
7	dry_air_low_humidity	low humidity	35.0	26.0	12.0	No
8	cool_humid_morning	low evapotranspiration	40.0	16.0	88.0	No
9	moderate_deficit_zone_c	dry soil	27.5	31.0	48.0	No
10	invalid_moisture_negative	invalid sensor input	-5.0	27.0	55.0	Yes
11	invalid_moisture_out_of_range	invalid sensor input	145.0	27.0	55.0	Yes
12	invalid_temperature_open_circuit	invalid sensor input	30.0	-40.0	55.0	Yes
13	invalid_humidity_unreadable	invalid sensor input	22.0	29.0	n/a	Yes

What assumptions are made?

- The moisture target is 45% volumetric water content.

- Valid ranges are 0-100% moisture, -10 to 60 C temperature and 0-100% relative humidity.
- The evapotranspiration factor is referenced to 25 C and 60% RH and clamped to 0.5-2.0.
- Base watering time is 1.25 minutes per percentage point of moisture deficit and valve time is capped at 60 minutes.
- Emitter flow is 0.125 L/min; 5 L is reserved in the rainwater tank; the pump is deferred below 30% battery.
- Processor clock is 100 MHz, with 32 registers and a 16-entry two-bit predictor in the optimized configuration.
- The processor-energy model is comparative and activity-based; absolute picojoule values are model assumptions, while relative comparisons are the intended result.

What constraints must be satisfied?

The implementation must remain deterministic, must not bypass the processor by making irrigation decisions in host-language conditionals, and must preserve the same memory results across all configurations. Multi-cycle arithmetic must occupy EX for its real iteration count, load-use dependencies must stall when required, branches must resolve consistently, and sensor faults must result in a safe valve-off decision.

2. Application of Course Knowledge

2.1 Sensor Data Representation - Q8.8 Fixed Point

Controller arithmetic uses signed 16-bit Q8.8 fixed point: eight integer/sign bits and eight fractional bits. A real value x is stored as $\text{round}(x \times 256)$. The resolution is $1/256 = 0.00390625$. For example, 45.0 is stored as raw integer 11520, binary 00101101.00000000 and hexadecimal 0x2D00. Q8.8 is appropriate because the sensor precision does not require a full floating-point datapath, while integer add/subtract can be reused for fixed-point values.

Example	Raw Q8.8	Hex	Binary
18.0% moisture	4608	0x1200	00010010.00000000
44.75% moisture	11456	0x2CC0	00101100.11000000
45.0 target	11520	0x2D00	00101101.00000000

2.2 IEEE 754 Binary32

For CO2 comparison, the project includes a custom IEEE 754 binary32 encoder/decoder. It explicitly constructs the sign, biased exponent and 23-bit fraction, supports zero, subnormal values, infinity and NaN, and uses round-to-nearest-even. For decimal 45.0 the encoded pattern is 0x42340000 with sign 0, exponent field 10000100 and mantissa field 011010000000000000000000. The

implementation is tested against Python's binary packing behavior over known and random patterns (IEEE 2019).

2.3 Booth Radix-2 Multiplication

Booth multiplication is used by the MULQ instruction. Each iteration inspects (Q0, Q-1): 01 adds the multiplicand, 10 subtracts it, and 00/11 perform no arithmetic, followed by an arithmetic right shift. A 16-bit Q8.8 multiply therefore occupies the EX stage for 16 iterations. The implementation uses a guard bit in the accumulator so the most-negative operand edge case remains correct (Booth 1951).

2.4 Restoring and Non-Restoring Division

Both sequential divider algorithms are implemented and validated. Restoring division subtracts the divisor after the left shift and restores the partial remainder when it becomes negative. Non-restoring division alternates subtract/add according to the sign of the partial remainder and performs a final correction if needed. DIVQ uses non-restoring division because each iteration performs exactly one add or subtract, giving a predictable multi-cycle EX latency (Mano 1993).

2.5 Five-Stage Pipelined Datapath

Stage	Function
IF	Fetch instruction from instruction memory and advance/predict PC.
ID	Decode opcode, read registers, identify source/destination registers, detect hazards.
EX	ALU operations, Booth multiply, divide, address calculation and branch resolution.
MEM	Read/write data memory or pass an ALU result.
WB	Write register result and retire the instruction.

Pipeline stages are evaluated WB -> MEM -> EX -> ID -> IF using snapshots of the current latches, so each stage observes the start-of-cycle state. Registers are written in the first half of a cycle and may be read by ID in the second half. A branch resolves at the end of EX, so a wrong-path redirect can cost two younger instructions (Patterson and Hennessy 2020).

2.6 Data, Structural and Control Hazards

Hazard	Detected/ Resolved	Handling
RAW data hazard	ID	Without forwarding the consumer waits for register write-back; with forwarding EX/MEM or MEM/WB values feed EX.

Hazard	Detected/ Resolved	Handling
Load-use hazard	ID	One bubble remains because a load value becomes available only after MEM.
Unified-memory structural hazard	IF/MEM	A LOAD/STORE in MEM blocks IF when a single memory port is modeled; split I/D memory removes the conflict.
Multi-cycle functional-unit hazard	EX	MULQ/DIVQ hold EX for the actual algorithm iteration count.
Control hazard	EX	Either freeze fetch, predict static not-taken, or use the two-bit dynamic predictor and squash on a miss.

2.7 Branch Prediction

The optimized processor uses a direct-mapped table of 16 two-bit saturating counters indexed by PC mod 16. Counters begin weakly not taken. States 0-1 predict not taken; states 2-3 predict taken. A single unexpected outcome only weakens the state instead of reversing the prediction immediately, which is effective for the loop back-edge and repeated irrigation branches (Patterson and Hennessy 2020).

2.8 Alternative Processor Techniques

Technique	Engineering assessment
Sequential execution	Simple and correct, but no stage overlap; used as the baseline.
In-order five-stage pipeline	Chosen implementation: transparent hazard behavior and suitable complexity for an embedded controller.
Superscalar issue	Could issue multiple independent instructions per cycle, but requires duplicated resources and dependency checking.
Out-of-order execution	Could hide long MUL/DIV latency but substantially increases control complexity, area and power.
SMT	Useful when multiple independent threads exist; the irrigation controller is a single deterministic control stream, so SMT is not selected.

3. Solution / Design / Methodology

3.1 System Architecture

The solution connects a realistic sensor-driven irrigation workload to a configurable processor simulator. Sensor values are conditioned, assembled into the controller's data memory, processed by the instruction program, and checked against an independent software reference. The same program is executed under six architecture configurations so performance improvements can be attributed to individual hardware features rather than to changes in the application logic.

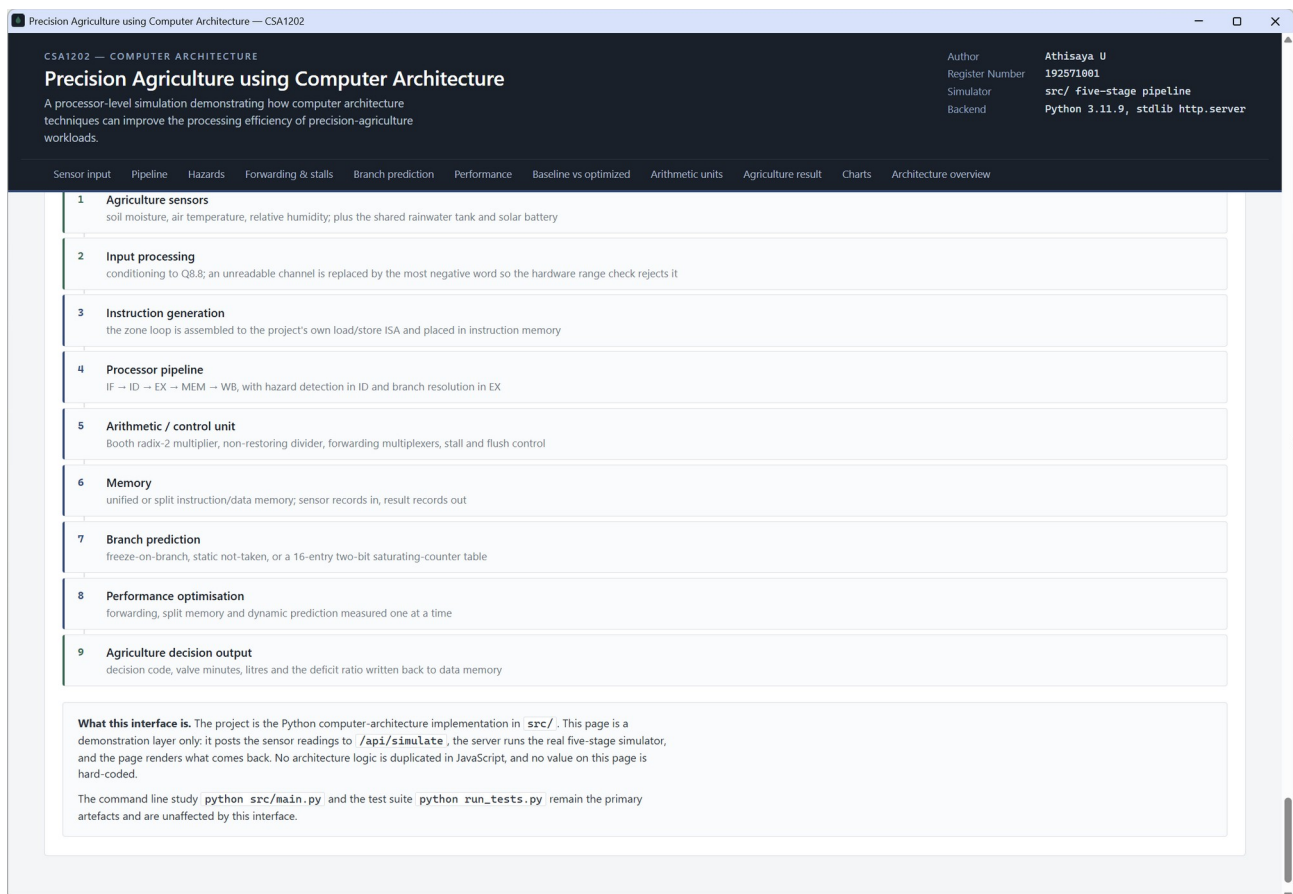


Figure 1. Web-demonstration architecture overview: sensors -> conditioning -> instruction program -> pipeline -> arithmetic/memory/prediction -> irrigation decision.

3.2 Irrigation Control Method

1. Read soil moisture, temperature and humidity for the current zone.
2. Convert readings to Q8.8 words and range-check them. Any invalid/unreadable sensor branches to a safe `SENSOR_FAULT_NO_IRRIGATION` decision.
3. Compute moisture deficit = target moisture - measured moisture. If deficit ≤ 0 , keep the valve closed.
4. Compute the evapotranspiration factor from temperature and humidity and clamp it to the configured range.
5. Compute valve time using Booth-based Q8.8 multiplication and cap it at 60 minutes.

6. Compute required water volume from valve time and emitter flow.
7. Check the shared solar battery. If below 30%, defer watering.
8. Check usable rainwater (tank minus 5 L reserve). If insufficient, trim the dose and recompute valve time using non-restoring division.
9. Store decision, valve minutes, litres and diagnostic deficit ratio, then update tank level and the running total.

3.3 Processor Configuration Study

Configuration	Forwarding	Memory	Branch handling
Sequential (non-pipelined)	no	split I/D	stall until EX
Basic pipeline	no	unified	stall until EX
Pipeline + forwarding	yes	unified	stall until EX
Pipeline + forwarding + split I/D memory	yes	split I/D	stall until EX
Pipeline + forwarding + split memory + static not-taken	yes	split I/D	static not-taken
Optimized pipeline (2-bit predictor)	yes	split I/D	2-bit dynamic

3.4 Pseudocode - Irrigation Controller

Controller pseudocode

```

1 INPUT: sensor cases, rainwater tank level, solar battery level
2 INITIALIZE total_litres = 0
3 FOR each zone
4   read moisture, temperature, humidity
5   convert readings to Q8.8
6   IF any sensor is invalid
7     decision = SENSOR_FAULT_NO_IRRIGATION
8     minutes = 0; litres = 0
9     CONTINUE
10  END IF

```

```
1 INPUT: sensor cases, rainwater tank level, solar battery level
11 deficit = moisture_target - moisture
12 IF deficit <= 0
13     decision = NO_IRRIGATION_SOIL_WET
14     minutes = 0; litres = 0
15     CONTINUE
16 END IF
17 ET = 1 + k_temp*(temperature-25) + k_humidity*(60-humidity)
18 clamp ET to [0.5, 2.0]
19 minutes = deficit * minutes_per_percent * ET
20 clamp minutes to 60
21 litres = minutes * flow_rate
22 IF battery < 30%
23     decision = DEFER_LOW_SOLAR_BATTERY
24     minutes = 0; litres = 0
25 ELSE
26     available = tank_litres - reserve
27     IF available <= 0
28         decision = DEFER_TANK_EMPTY
29         minutes = 0; litres = 0
30     ELSE IF litres > available
31         litres = available
32         minutes = litres / flow_rate
33         decision = IRRIGATE_REDUCED_TANK_LIMIT
34     ELSE
35         decision = IRRIGATE_FULL
36     END IF
37 END IF
```



```

1 INPUT: sensor cases, rainwater tank level, solar battery level
38 tank_litres = tank_litres - litres
39 total_litres = total_litres + litres
40 STORE decision, minutes, litres, deficit_ratio
41 END FOR
42 RETURN stored results

```

3.5 Pseudocode - Pipeline Cycle

Cycle-level processor pseudocode

1	ON EACH CLOCK CYCLE
2	snapshot current IF/ID, ID/EX, EX/MEM and MEM/WB latches
3	WB: write register result and retire instruction
4	MEM: perform data-memory access and build next MEM/WB latch
5	EX: apply forwarding; run ALU/MUL/DIV; resolve branch
6	ID: decode; detect RAW/load-use hazards; build next ID/EX latch
7	IF: fetch unless blocked by branch, memory-port or decode stall
8	IF prediction was wrong: squash younger instructions and redirect PC
9	atomically replace current latches with next latches
10	REPEAT until HALT retires

4. Use of Modern Tools

Tool	Use
Python 3.11.9	Cycle-accurate processor simulator, assembler, arithmetic algorithms, reference model, metrics and automated tests.
unittest	Final suite of 186 deterministic unit, integration, API and system tests.
Git / GitHub	Version control and public repository for source, results and evidence.
HTML5 / CSS3 / JavaScript	Browser-based presentation layer that visualizes simulator JSON; no processor algorithm is duplicated in JavaScript.

Tool	Use
Python stdlib http.server	Zero-dependency local server for the demonstration dashboard.
CSV / JSON	Machine-readable sensor cases, scenarios, cycle traces, hazard reports and performance results.
Web browser	Interactive demonstration of sensor inputs, pipeline cycles, hazards, prediction, arithmetic and irrigation results.

4.1 Software and Hardware Configuration

Item	Configuration
Operating environment	Windows 11 development laptop
Python	3.11.9
Project dependencies	Python standard library only for the simulator and web server
Dashboard backend	Python stdlib http.server
Dashboard frontend	HTML5, CSS3 and JavaScript
Local dashboard URL	http://127.0.0.1:8000/
Run commands	python src/main.py python run_tests.py python run_web.py
Additional hardware	None required; sensors and processor are simulated deterministically

5. Results and Validation

5.1 Automated Test Validation

The final test log reports 186 tests executed with 0 failures, 0 errors and 0 skipped tests. The suite includes the original processor/arithmetic tests and additional web/API tests that verify dashboard values against direct simulator calls. Representative checks prove exact Booth products, agreement of both division algorithms, Q8.8 and IEEE 754 behavior, forwarding paths, load-use interlocks, structural hazards, branch predictor behavior, identical decisions across all six configurations, sensor-fault handling, water accounting and HTTP/API correctness.

Final test summary	Value
Tests run	186

Final test summary	Value
Failures	0
Errors	0
Skipped	0
Result	PASS

Final test log: Ran 186 tests in 3.275s - OK | failures 0 | errors 0 | skipped 0 | result PASS

5.2 Full 13-Case Processor Performance Study

Execution mode	Cycles	CPI	Time (us)	Throughput (MIPS)	Speedup
Sequential (non-pipelined)	5000	5.988	50.00	16.70	1.000x
Basic pipeline	2853	3.417	28.53	29.27	1.753x
Pipeline + forwarding	2139	2.562	21.39	39.04	2.338x
+ split I/D memory	1994	2.388	19.94	41.88	2.508x
+ static not-taken	1796	2.151	17.96	46.49	2.784x
Optimized 2-bit predictor	1746	2.091	17.46	47.82	2.864x

Additional performance and energy metrics

Execution mode	IPC	Avg. latency (cycles)	Stall cycles	Energy (pJ)	Energy saved
Sequential (non-pipelined)	0.167	5.988	0	85,619	0.00%
Basic pipeline	0.293	7.910	2,014	83,779	2.15%
Pipeline + forwarding	0.390	7.007	1,300	78,532	8.28%
+ split I/D memory	0.419	7.007	1,155	76,647	10.48%
+ static not-taken	0.465	7.007	851	76,723	10.39%
Optimized 2-bit predictor	0.478	7.007	851	75,479	11.84%

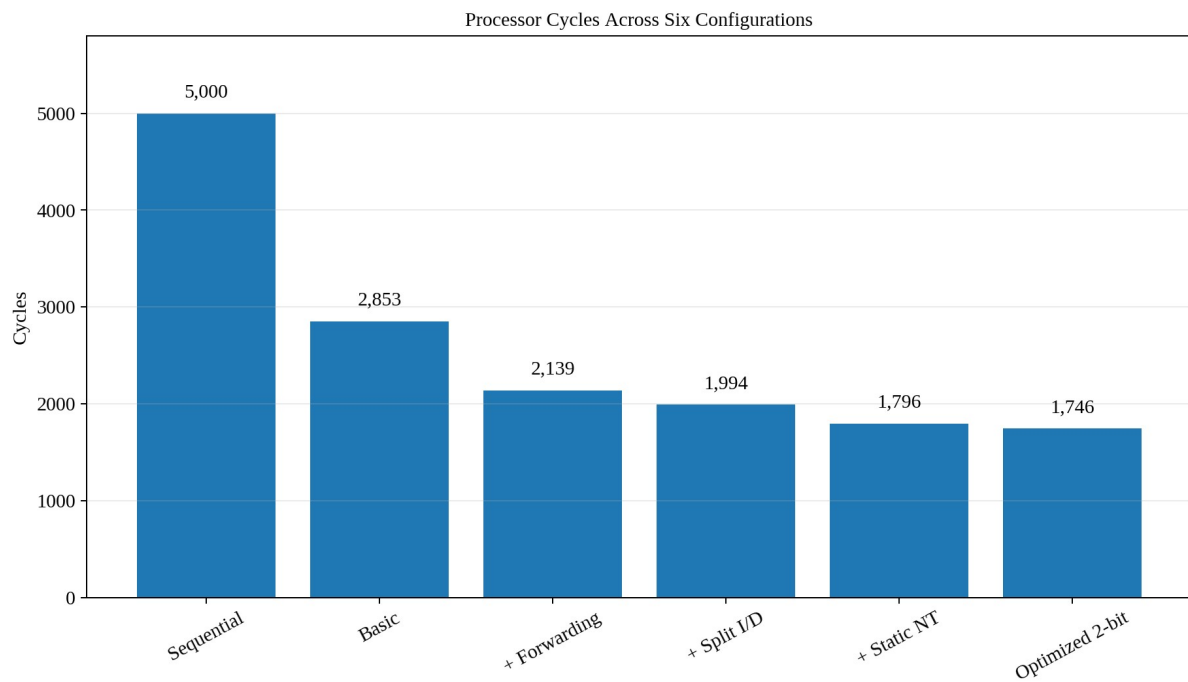


Figure 2. Processor cycle count across the six execution configurations for the full 13-case study.

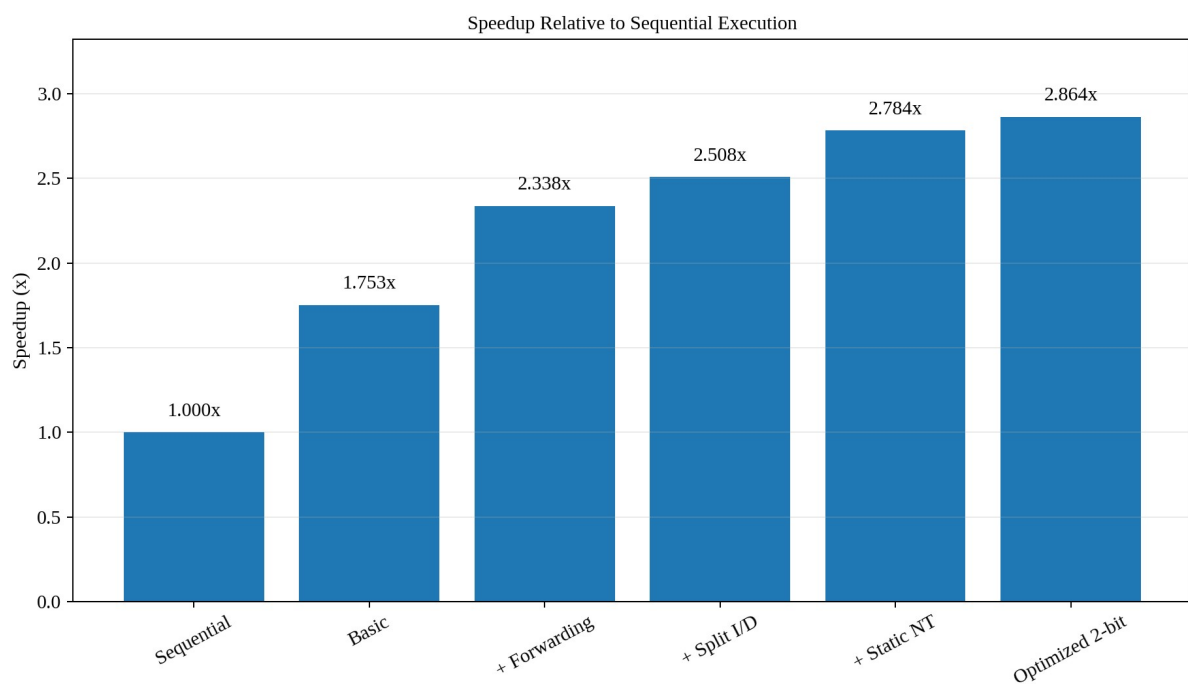


Figure 3. Speedup relative to sequential execution across the six processor configurations.

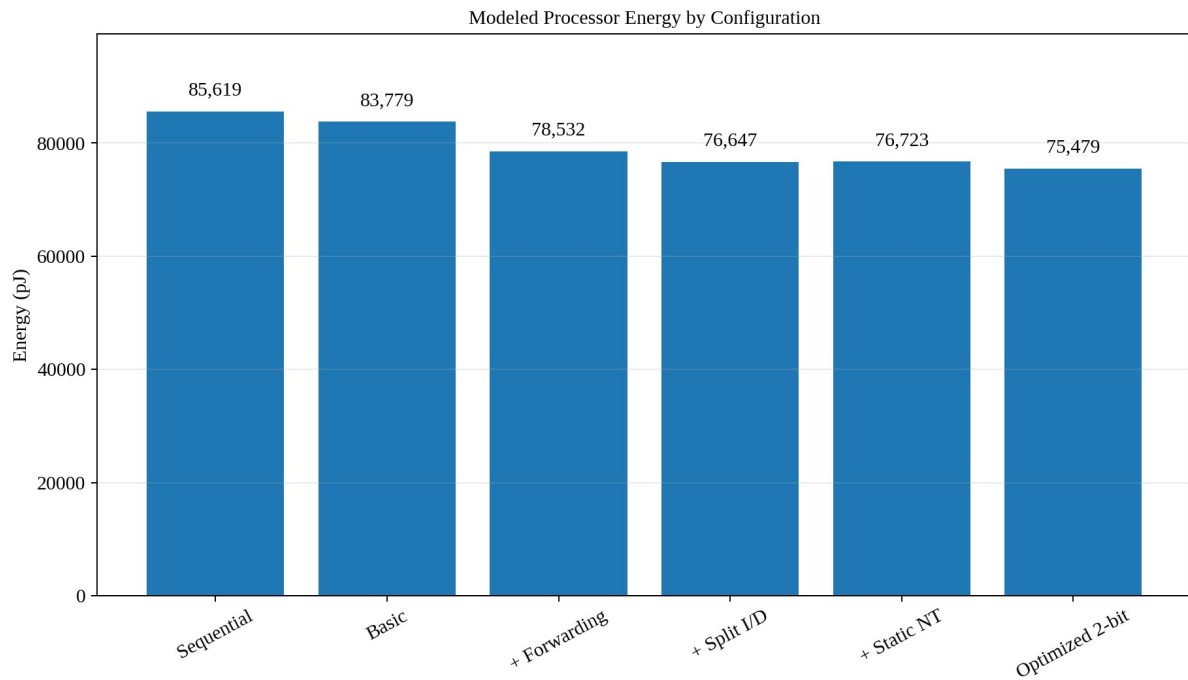


Figure 4. Modeled processor energy across the six execution configurations.

All configurations retire the same 835 instructions. The optimized pipeline completes the full 13-case study in 1,746 cycles instead of 5,000 sequential cycles, giving 2.863688x speedup. CPI falls from 5.988 to 2.091 and throughput increases from 16.70 MIPS to 47.82 MIPS at the modeled 100 MHz clock. Total processor energy decreases from 85,619 pJ to 75,479 pJ, an 11.8432% reduction.

5.3 Hazard and Branch-Prediction Results

Mode	RAW	Load-use	Memory structural	MUL/DIV structural	Control
Basic pipeline	728	52	105	825	304
+ forwarding	0	26	145	825	304
+ split I/D memory	0	26	0	825	304
+ static not-taken	0	26	0	825	0
Optimized 2-bit	0	26	0	825	0

Branch-prediction comparison

Prediction policy	Branches	Mispredictions	Squashed	Accuracy
Static not-taken	152	53	93	65.13%
2-bit dynamic	152	28	56	81.58%

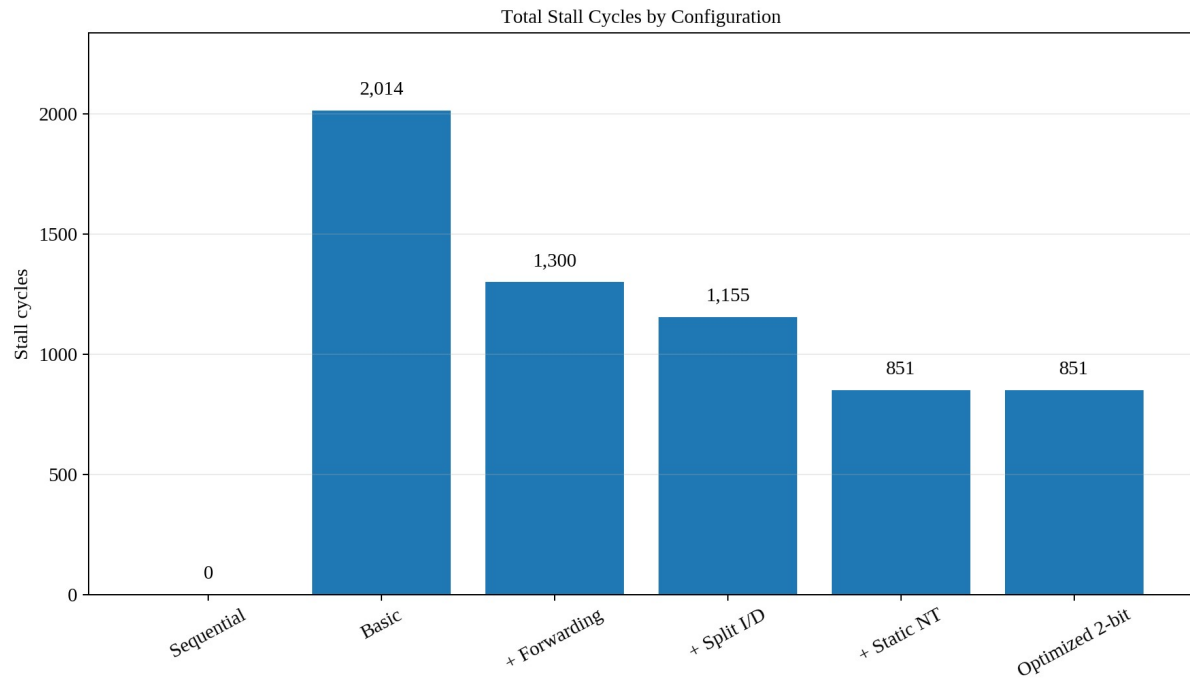


Figure 5. Total stall cycles as forwarding, split memory and branch prediction are introduced.

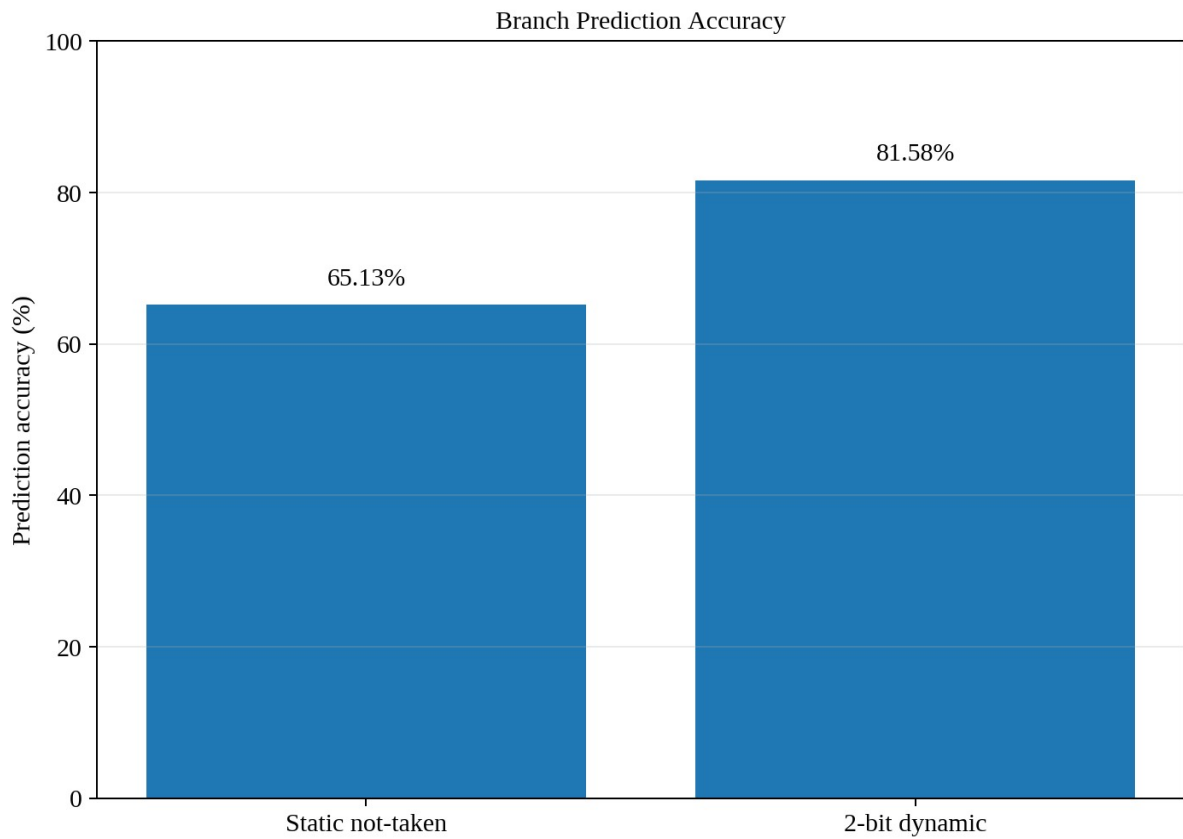


Figure 6. Branch-prediction accuracy for static not-taken and the 16-entry two-bit dynamic predictor.

Forwarding removes all ordinary RAW stalls (728 $\rightarrow$ 0) and leaves only 26 load-use stalls. Split instruction/data memory removes structural memory conflicts entirely. Static not-taken predicts 65.13% of 152 executed branches correctly, while the two-bit dynamic predictor reaches 81.58% accuracy and reduces mispredictions from 53 to 28. The remaining 825 functional-unit stall cycles are deliberate multi-cycle Booth/division occupancy, which explains why the measured speedup is below the ideal five-stage limit.

5.4 Irrigation and Resource Results

Scenario	Fixed timer	Controller	Reduction	Interpretation
A - full tank, healthy battery	48.7500 L	24.3438 L	50.06%	Primary water-saving comparison; sensor control avoids unnecessary watering.
B - low solar battery	48.7500 L	0.0000 L	100.00% deferred	Irrigation is postponed because stored energy is insufficient.

Scenario	Fixed timer	Controller	Reduction	Interpretation
C - nearly empty tank	48.7500 L	13.0000 L	73.33% lower	Part of the reduction is caused by the water-supply limit and 5 L reserve.

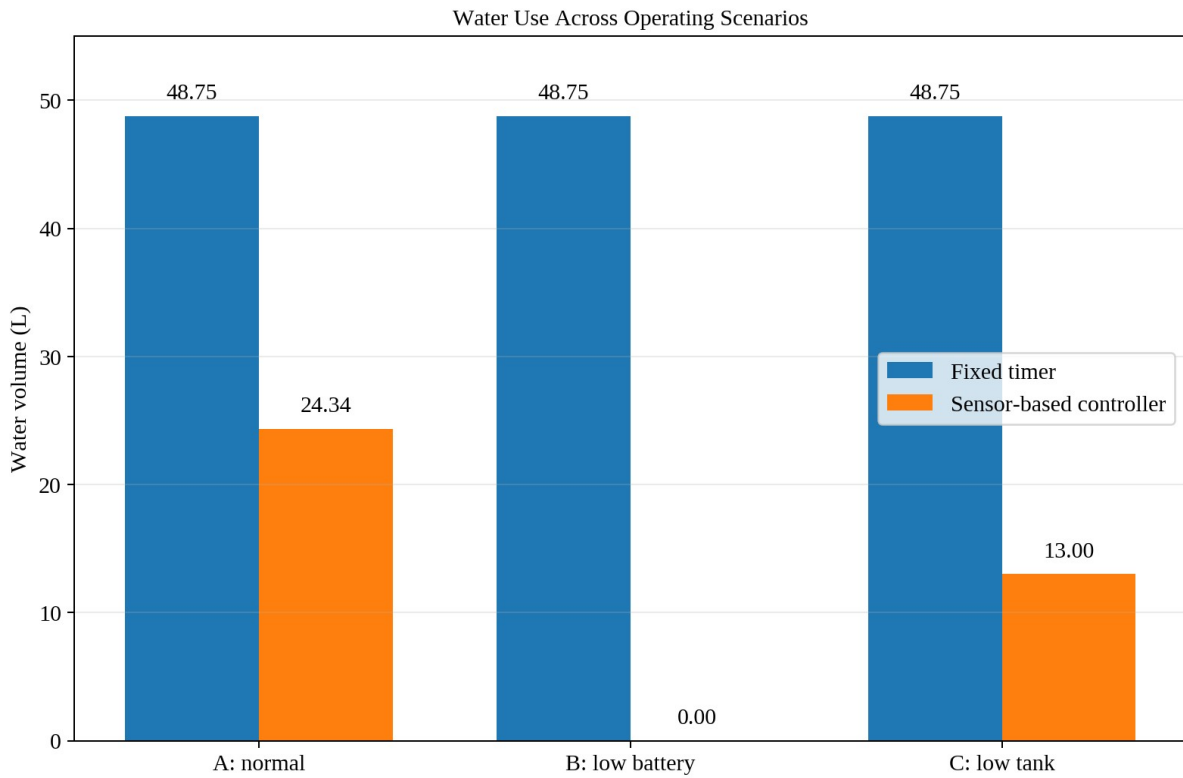


Figure 7. Fixed-timer and sensor-based water use across the three operating scenarios.

The key causal distinction is maintained throughout the project: sensor-based decision logic is what changes water use, whereas pipeline optimization changes computation time and processor energy. Tests explicitly verify that pipelining does not change a single litre; sequential, basic and optimized processors execute the same control program and produce byte-identical decisions.

5.5 Web Demonstration Screenshots

The dashboard is a visualization layer over the existing Python simulator. `POST /api/simulate` runs the processor configurations and returns cycle traces, hazards, forwarding, branch statistics, performance, energy and irrigation decisions. The screenshots below are from the running local dashboard. Some figures show the default three-zone interactive demonstration (for example 1,355 -> 479 cycles); the headline 5,000 -> 1,746 result above is the separate full 13-case batch study.

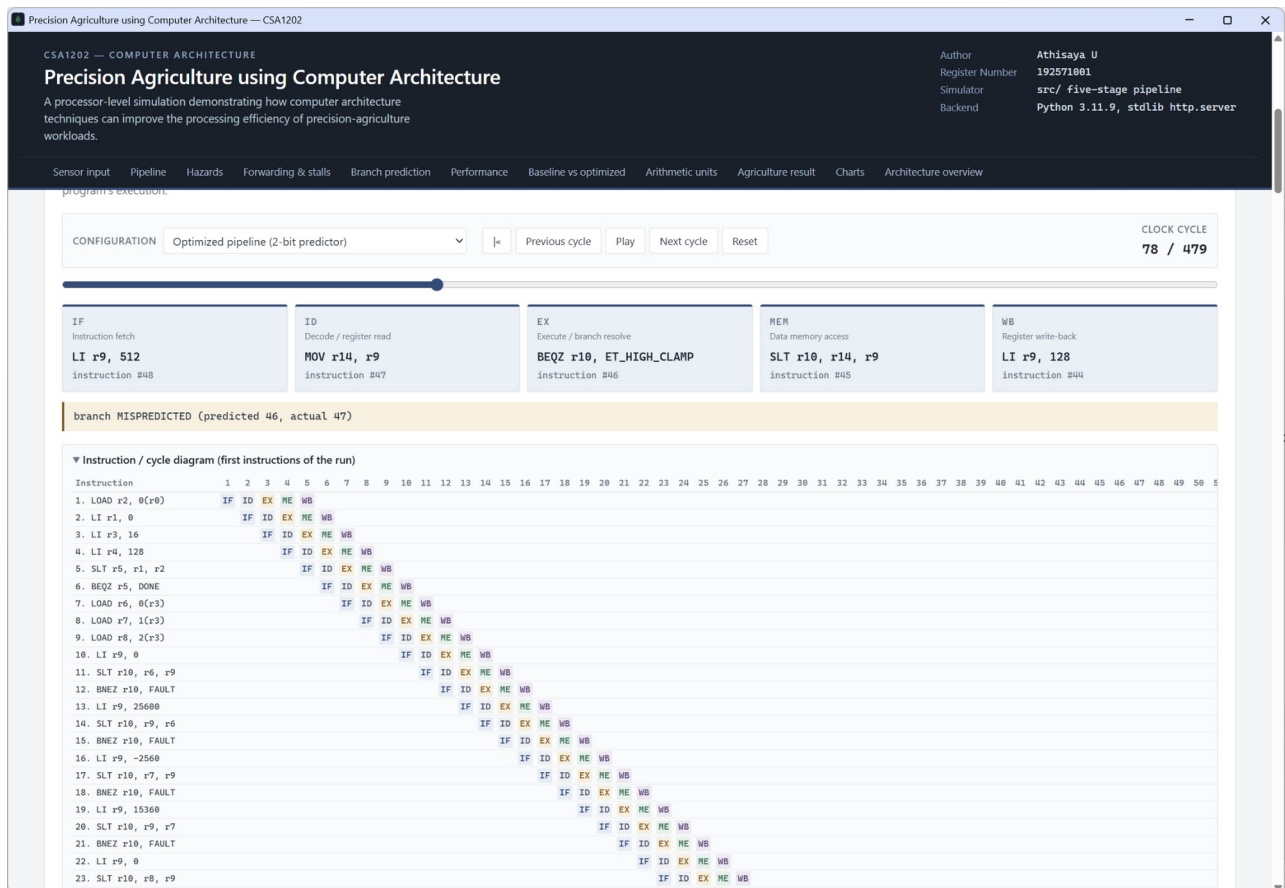


Figure 9. Cycle-by-cycle IF-ID-EX-MEM-WB visualization at a branch misprediction in the optimized pipeline.

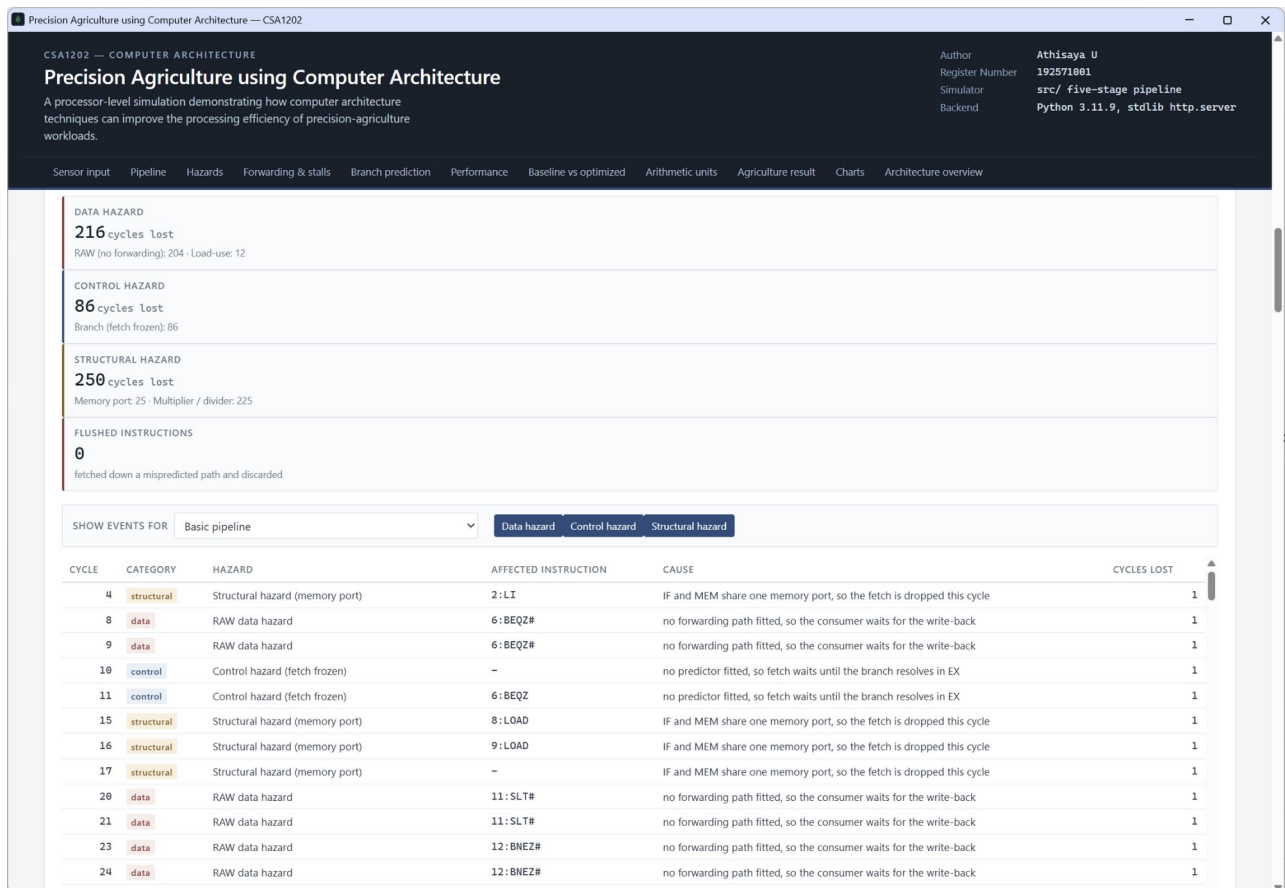


Figure 10. Hazard analysis showing data, control and structural events with affected instruction, cause and cycles lost.

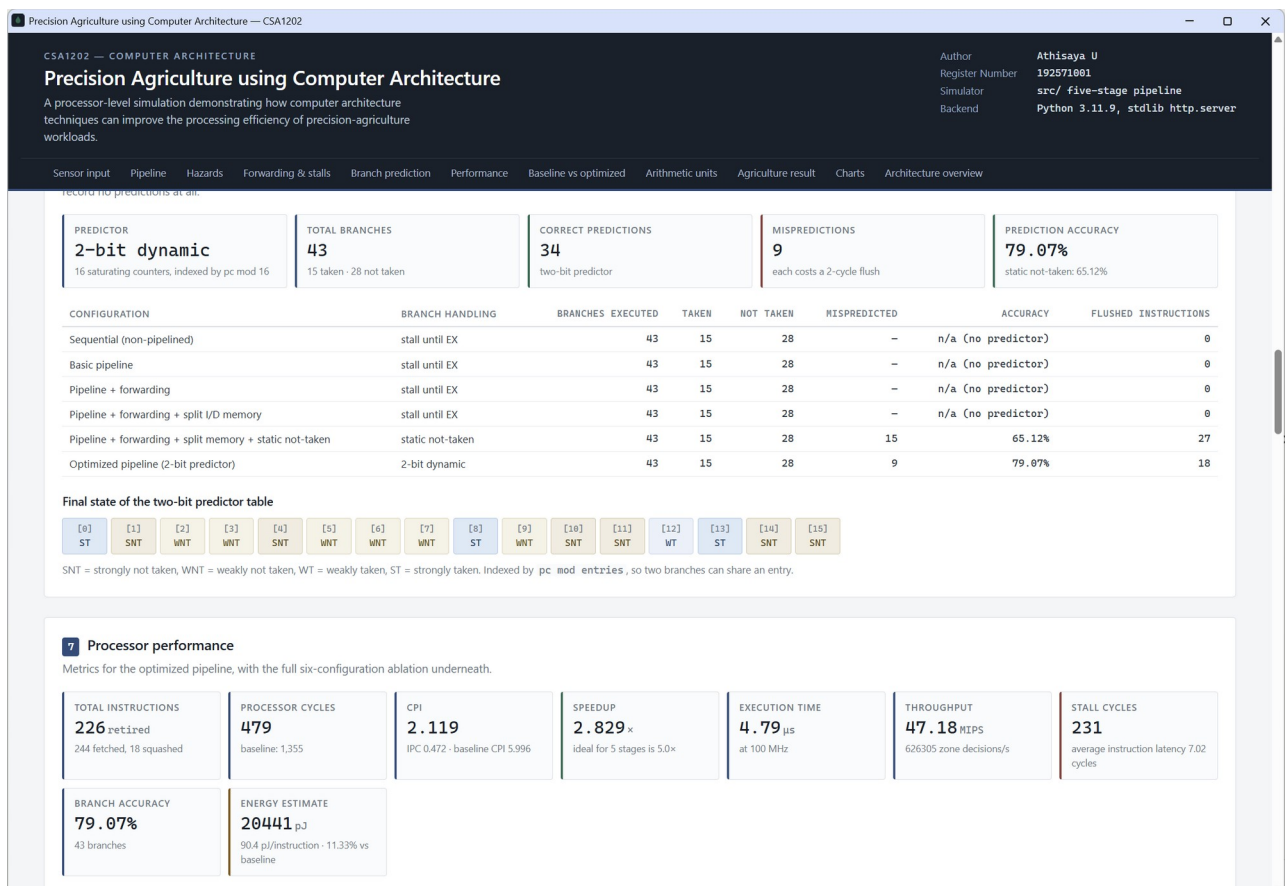


Figure 11. Branch-prediction dashboard showing static and two-bit dynamic predictor statistics.

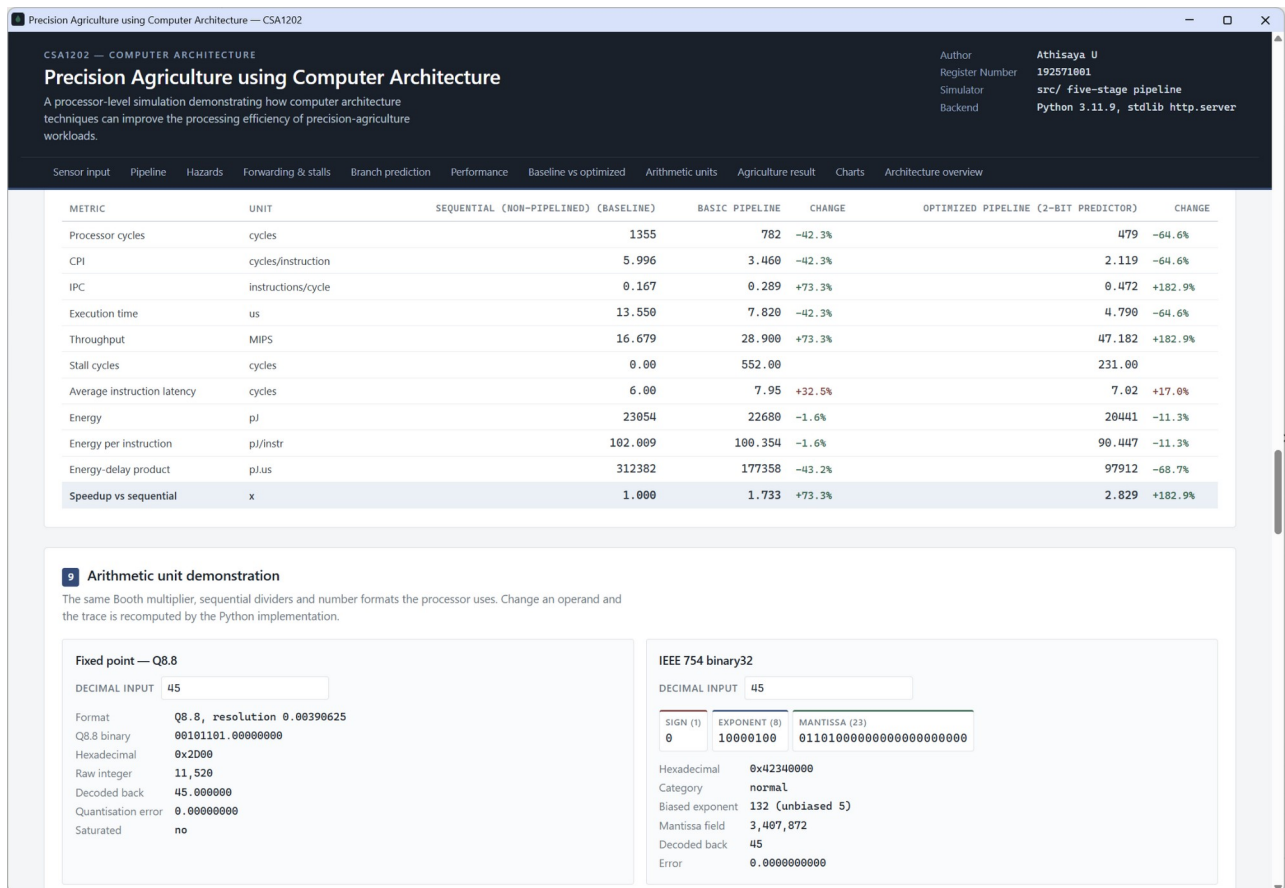


Figure 12. Interactive three-zone baseline-versus-optimized performance metrics; these values belong to the dashboard demonstration run.

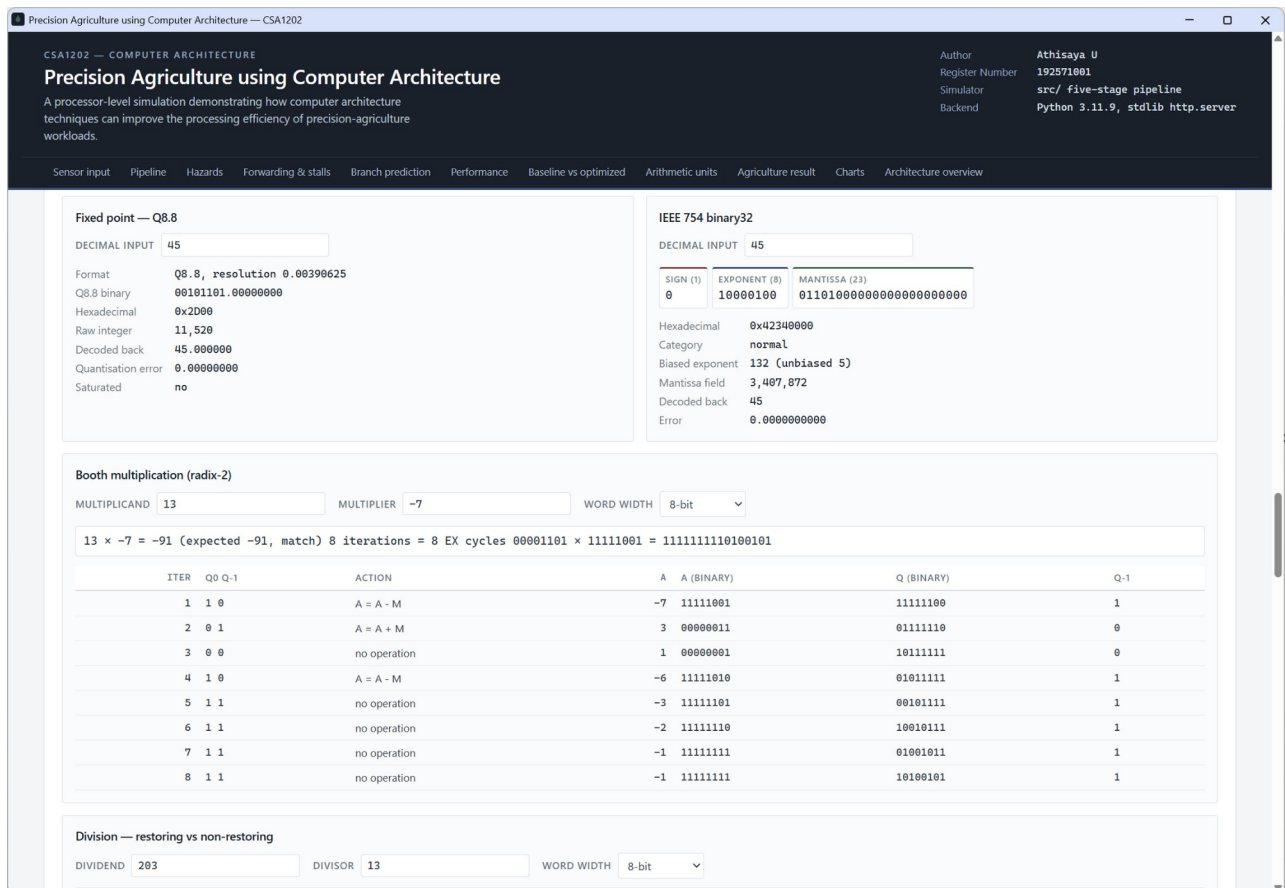


Figure 13. Q8.8, IEEE 754 and Booth arithmetic demonstrations computed by the Python arithmetic modules used by the processor.

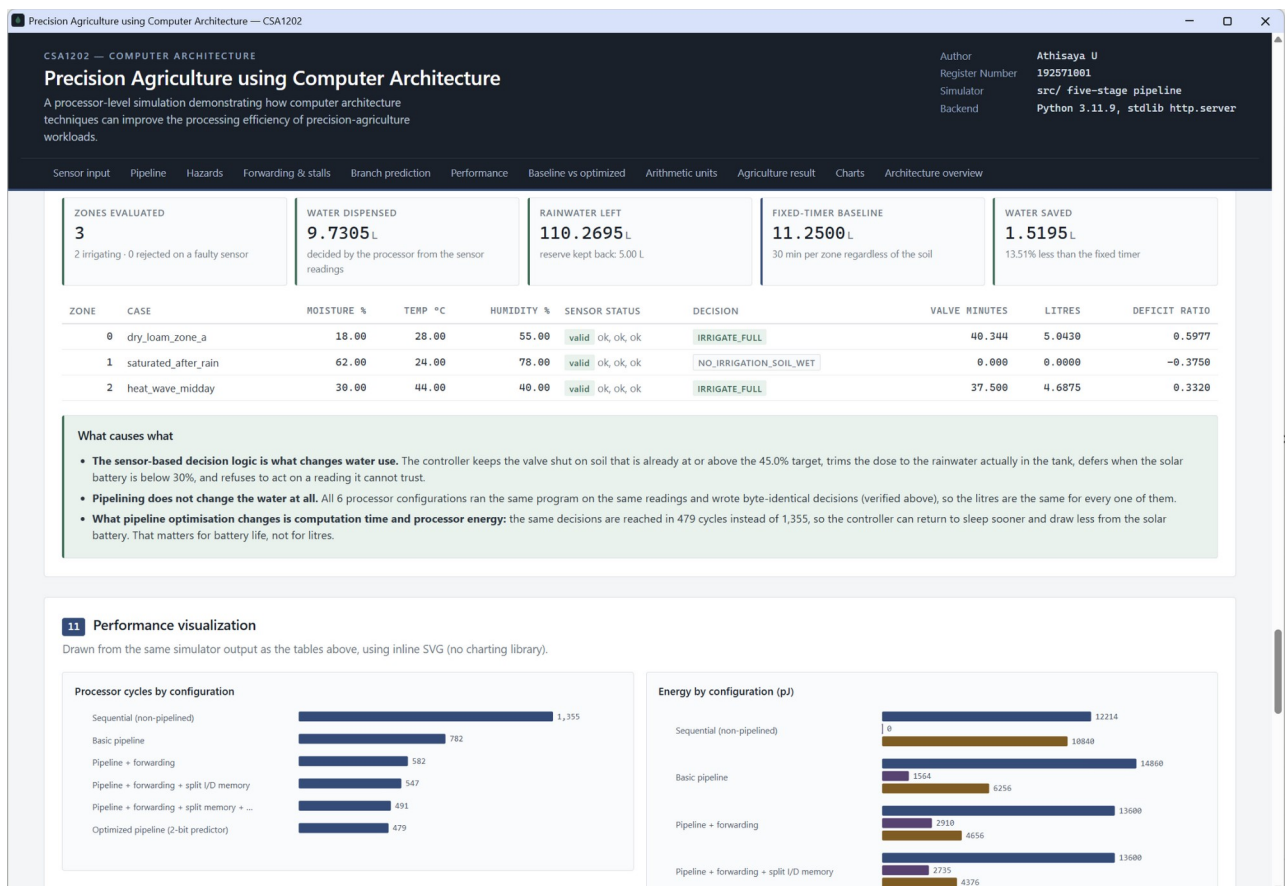


Figure 14. Interactive three-zone irrigation decisions, rainwater state and fixed-timer comparison.



Figure 15. Performance visualization generated from the simulator payload for the interactive run.

6. Analysis and Engineering Decision

The measurements show that the final performance is the cumulative result of several architectural decisions rather than a single optimization. The basic pipeline overlaps stages but still loses 2,014 cycles to hazards. Adding forwarding produces the largest post-pipeline improvement by eliminating ordinary RAW waits. Split I/D memory removes the fetch/data-memory resource conflict. Prediction then replaces control freezes with speculative execution; the two-bit predictor reduces wrong-path work compared with static not-taken.

Decision	Reason	Trade-off / evidence
Q8.8 instead of floating point	Chosen for deterministic sensor arithmetic, 0.00390625 resolution and simpler embedded datapath. IEEE 754 is retained as a CO2 demonstration/comparison.	Lower hardware cost and predictable arithmetic; smaller numeric range than binary32.

Decision	Reason	Trade-off / evidence
Non-restoring DIVQ	Chosen for the processor instruction because it performs one add/subtract per iteration. Restoring division is still implemented and tested.	Predictable multi-cycle latency; still occupies EX for many cycles.
Forwarding	Chosen to resolve ALU dependencies without waiting for WB.	Extra bypass paths/control logic, but removes 728 measured RAW stalls.
Split I/D memory	Chosen after forwarding to remove unified-port conflicts.	Uses separate instruction/data paths but eliminates measured memory structural stalls.
Two-bit predictor	Chosen over static not-taken for repeated branch patterns.	Adds predictor state and update logic; accuracy improves from 65.13% to 81.58%.
In-order five-stage pipeline	Chosen over superscalar/OoO/SMT for clarity, deterministic behavior and embedded energy focus.	Long MUL/DIV latency remains visible; more aggressive designs could hide it at higher area/power complexity.

The optimized CPI remains 2.091 rather than approaching 1 because multi-cycle arithmetic dominates the residual stalls. That is a modeled hardware consequence, not a correctness defect. A future design could pipeline or duplicate multiply/divide resources, but that would increase area and dynamic switching energy. For a small solar-powered controller that sleeps between sensing intervals, the selected architecture provides a strong improvement without making the control hardware unnecessarily complex.

The water result is interpreted conservatively. Scenario A is the defensible water-saving comparison because both battery and tank are healthy. Scenario B defers irrigation rather than eliminating future demand, and Scenario C is partially constrained by water supply. Therefore the 50.06% reduction in Scenario A is the primary sustainability result, while the 2.864x speedup and 11.84% processor-energy reduction are the primary computer-architecture results.

7. Broader Considerations

Consideration	Discussion
Sustainability	Sensor-driven irrigation avoids watering already-wet soil and respects a rainwater reserve. The normal scenario uses 24.3438 L instead of 48.75 L, a 50.06% reduction.
Energy	The optimized processor finishes the same workload in fewer cycles and reduces modeled processor energy by 11.84%, useful for a solar-powered controller.
Safety	Out-of-range or unreadable sensors produce a safe no-irrigation decision; low battery and unavailable water also defer the pump.
Reliability	Every architecture configuration is checked against an independent software reference and the final suite reports 186/186 tests passing.
Societal relevance	Reliable precision irrigation supports efficient crop-water management and is aligned with SDG 2 - Zero Hunger through improved agricultural resource use.
Economics	The prototype is software-only and uses Python standard-library tools, so no special hardware or paid software is required for evaluation.
Ethics / professional responsibility	Performance and water claims are separated by cause; constrained/deferred scenarios are not misrepresented as permanent water savings.
Limitations	Agricultural constants are illustrative controller settings and require field calibration for a real crop. The energy model is comparative rather than transistor-level power measurement.
Possible improvements	Integrate real ADC/sensor readings, persistent state across irrigation intervals, weather forecasts, and a hardware experiment with pipelined/parallel multiply-divide resources.

8. Conclusion

The assignment successfully implements a precision-agriculture irrigation workload on a cycle-accurate five-stage computer-architecture simulator. It combines Q8.8 fixed point, IEEE 754 binary32 demonstration, Booth radix-2 multiplication, restoring and non-restoring division, a

load/store instruction set, explicit pipeline latches, forwarding, interlocks, structural-hazard handling, static prediction and a dynamic two-bit branch predictor.

The full 13-case study retires 835 instructions. The sequential baseline requires 5,000 cycles, while the optimized pipeline requires 1,746 cycles, giving a 2.863688x speedup. CPI decreases from 5.988 to 2.091, throughput increases from 16.70 to 47.82 MIPS, and modeled processor energy falls by 11.8432%. The dynamic predictor reaches 81.58% accuracy. All six processor configurations produce identical irrigation decisions and match the independent reference model.

Under the normal full-tank, healthy-battery scenario, sensor-based irrigation uses 24.3438 L compared with 48.75 L for the fixed timer, a 50.06% reduction. This saving comes from the control policy, while pipelining improves computation time and processor energy. The final web interface exposes these same calculations interactively without replacing the Python computer-architecture implementation.

9. Student Reflection

What did you learn from this assignment beyond what was directly taught in the classroom?

This assignment helped me understand that a pipeline is not only a five-box diagram. I had to connect every stage to real instruction behavior and then see what happens when a value is not ready, when the instruction and data sides compete for a resource, or when a branch changes the PC after younger instructions have already entered the pipeline. The cycle trace made forwarding, bubbles, squashing and multi-cycle execution much easier to understand because I could observe the exact instruction occupying IF, ID, EX, MEM and WB at each clock cycle.

I also learned why the arithmetic algorithm matters to the architecture. Booth multiplication and non-restoring division are not isolated textbook procedures in this project: the number of iterations directly becomes EX-stage latency. That connection explains why a five-stage processor does not automatically give a five-times speedup. I also gained a clearer understanding of Q8.8 fixed point, saturation and quantisation, and how IEEE 754 represents the same value using sign, exponent and mantissa fields.

The validation work was equally important. Comparing all six processor configurations with an independent reference model taught me to separate functional correctness from performance. A faster pipeline is useful only if it produces the same irrigation decisions. The final 186-test suite also showed me the value of testing boundary conditions, faults, divide-by-zero behavior, branch state transitions, HTTP errors and even cases such as a browser closing during a response.

Building the web demonstration improved how I present technical work. Instead of replacing the simulator with a web application, I learned to keep the Python architecture as the source of truth and let HTML, CSS and JavaScript visualize its results. The pipeline cycle controls, hazard table, branch

predictor state, arithmetic traces and performance charts now make the internal behavior easier to explain during evaluation while keeping the processor implementation unchanged.

If you were given additional time/resources, what would you improve and why?

I would connect the controller to real sensor/ADC readings and calibrate moisture, evapotranspiration and emitter-flow constants for an actual crop and soil type. I would also compare the current in-order design with a pipelined or duplicated multiply/divide unit, and possibly a limited superscalar experiment, to study whether the additional hardware complexity is justified by the remaining 825 functional-unit stall cycles. For the application side, persistent multi-day irrigation state and weather-forecast input would make the agriculture model more realistic while keeping the safety checks deterministic.

10. References

- Booth, Andrew D. 1951. "A Signed Binary Multiplication Technique." *The Quarterly Journal of Mechanics and Applied Mathematics* 4 (2): 236-240. <https://doi.org/10.1093/qjmam/4.2.236>.
- Hamouda, Fatma, Àngela Puig-Sirera, Lorenzo Bonzi, Damiano Remorini, Rossano Massai, and Giovanni Rallo. 2024. "Design and Validation of a Soil Moisture-Based Wireless Sensors Network for the Smart Irrigation of a Pear Orchard." *Agricultural Water Management* 305: 109138. <https://doi.org/10.1016/j.agwat.2024.109138>.
- IEEE. 2019. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019. New York: Institute of Electrical and Electronics Engineers. <https://doi.org/10.1109/IEEESTD.2019.8766229>.
- Mano, M. Morris. 1993. *Computer System Architecture*. 3rd ed. Englewood Cliffs, NJ: Prentice Hall.
- Monteiro, António, Sérgio Santos, and Pedro Gonçalves. 2021. "Precision Agriculture for Crop and Livestock Farming-Brief Review." *Animals* 11 (8): 2345. <https://doi.org/10.3390/ani11082345>.
- Patterson, David A., and John L. Hennessy. 2020. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. 2nd ed. Cambridge, MA: Morgan Kaufmann.

Project Repository and Reproduction

GitHub: <https://github.com/athisayaprema2007-hue/CSA1202-Precision-Agriculture-Pipeline>

Run commands

1	<code>python src/main.py</code>
2	<code>python run_tests.py</code>
3	<code>python run_web.py</code>
4	# Open <code>http://127.0.0.1:8000/</code> in the browser

Appendix A - Core Source Code (Text Format)

The complete repository contains all source, tests, data and result files. The listings below reproduce the core implementation paths directly from the submitted project with line numbers. The web demonstration remains a presentation layer; the computer-architecture algorithms are in src/.

src/fixed_point.py - Q8.8 conversion, multiplication and division

```
38 SENSOR_FAULT_RAW = Q_MIN_RAW
39
40
41 def saturate(raw: int) -> int:
42     """Clamp a raw value into the Q8.8 range instead of letting it wrap."""
43     if raw > Q_MAX_RAW:
44         return Q_MAX_RAW
45     if raw < Q_MIN_RAW:
46         return Q_MIN_RAW
47     return raw
48
49
50 def to_q88(value: float) -> int:
51     """Convert a real number to its raw Q8.8 representation (saturating)."""
52     return saturate(int(round(value * SCALE)))
53
54
55 def from_q88(raw: int) -> float:
56     """Convert a raw Q8.8 value back to a real number."""
57     return raw / SCALE
58
59
60 def q_bits(raw: int) -> str:
61     """16-bit two's-complement bit pattern, split at the binary point."""
```

```

62     pattern = format(to_unsigned(raw, WORD_BITS), "016b")
63     return pattern[:8] + "." + pattern[8:]
64
65
66 def q_hex(raw: int) -> str:
67     return "0x{:04X}".format(to_unsigned(raw, WORD_BITS))
68
69
70 def q_format(raw: int, places: int = 4) -> str:
71     """e.g. ``45.0000 (0x2D00)`` -- used by the printed tables."""
72     return "{:.{p}f} ({{}})".format(from_q88(raw), q_hex(raw), p=places)
73
74
75 def q_add(a: int, b: int) -> int:
76     """Saturating Q8.8 addition (plain integer add on the raw values)."""
77     return saturate(a + b)
78
79
80 def q_sub(a: int, b: int) -> int:
81     """Saturating Q8.8 subtraction."""
82     return saturate(a - b)
83
84
85 def q_mul_traced(a: int, b: int) -> Tuple[int, int]:
86     """Q8.8 multiply using Booth's algorithm.
87
88     Q8.8 x Q8.8 gives a Q16.16 product, so the 32-bit Booth result is
89     rounded and shifted right by FRAC_BITS to get back to Q8.8.

```

```

90
91     Returns ``(raw_result, booth_iterations)``.
92     """
93     product, steps = booth_multiply(a, b, WORD_BITS)
94     # round-to-nearest by adding half an LSB before the arithmetic shift
95     rounded = (product + (1 << (FRAC_BITS - 1))) >> FRAC_BITS
96     return saturate(rounded), len(steps)
97
98
99 def q_mul(a: int, b: int) -> int:
100     return q_mul_traced(a, b)[0]
101
102
103 def q_div_traced(a: int, b: int) -> Tuple[int, int, bool, bool]:
104     """Q8.8 divide using the non-restoring divider.
105
106     The dividend is pre-shifted left by FRAC_BITS so that the quotient comes
107     out already in Q8.8 form.
108
109     Returns ``(raw_result, division_iterations, divide_by_zero, overflow)``.
110     """
111     if b == 0:
112         # Divide-by-zero saturates and raises a flag, it does not crash.
113         result = Q_MAX_RAW if a >= 0 else Q_MIN_RAW
114         return result, 0, True, True
115
116     dividend = a << FRAC_BITS
117     division = signed_divide(dividend, b, WORD_BITS, method="non_restoring")

```

```

118     quotient = division.quotient
119     overflow = division.overflow or quotient > Q_MAX_RAW or quotient <
Q_MIN_RAW
120     return saturate(quotient), division.iterations, False, overflow
121
122
123 def q_div(a: int, b: int) -> int:
124     return q_div_traced(a, b)[0]
125
126
127 def quantisation_error(value: float) -> float:
128     """How much precision a real number loses when stored as Q8.8."""
129     return abs(value - from_q88(to_q88(value)))

```

src/booth.py - Booth radix-2 multiplier

```

55 def booth_multiply(multiplicand: int, multiplier: int,
56                    bits: int = 16) -> Tuple[int, List[BoothStep]]:
57     """Signed multiply using Booth's radix-2 algorithm.
58
59     Registers follow the classic textbook datapath:
60
61     A : accumulator / partial product high half  (`bits` wide)
62     Q : multiplier, becomes the low half of the product
63     Q-1 : the extra "previous bit" flip-flop
64     M : multiplicand
65
66     Each iteration looks at the pair (Q0, Q-1):
67

```

```

68     0 0 / 1 1 -> do nothing      (inside a run of equal bits)
69     0 1      -> A = A + M      (end of a run of 1s)
70     1 0      -> A = A - M      (start of a run of 1s)
71
72     and then arithmetically shifts the triple (A, Q, Q-1) right by one place.
73
74     Returns ``(product, steps)`` where ``product`` is a signed ``2*bits``
75     value and ``steps`` has exactly ``bits`` entries.
76     """"
77     mask = (1 << bits) - 1
78     m = to_signed(multiplicand, bits)
79     a = 0
80     q = to_unsigned(multiplier, bits)
81     q_minus_1 = 0
82     steps: List[BoothStep] = []
83
84     for i in range(bits):
85         q0 = q & 1
86         q_minus_1_before = q_minus_1
87         # The adder is one bit wider than A. Without that guard bit the sum
88         # A + M can overflow, and the most negative operand times itself
89         # (for example -128 x -128 with 8-bit words) would come out negative.
90         if q0 == 0 and q_minus_1 == 1:
91             a = to_signed(a + m, bits + 1)
92             action = "A = A + M"
93         elif q0 == 1 and q_minus_1 == 0:
94             a = to_signed(a - m, bits + 1)
95             action = "A = A - M"

```

```

96     else:
97         action = "no operation"
98
99     # Arithmetic right shift of the combined (A, Q, Q-1) register.
100    q_minus_1 = q & 1
101    q = ((q >> 1) | ((a & 1) << (bits - 1))) & mask
102    a = a >> 1 # Python's >> on a negative int is already arithmetic
103
104    steps.append(BoothStep(i + 1, q0, q_minus_1_before,
105                          action, a, q, q_minus_1))
106
107    product = (to_unsigned(a, bits) << bits) | q
108    return to_signed(product, 2 * bits), steps
109
110
111 def booth_trace_table(multiplicand: int, multiplier: int, bits: int = 8) -> str:
112     """Human-readable trace of a Booth multiplication (used by the report)."""
113     product, steps = booth_multiply(multiplicand, multiplier, bits)
114     lines = []
115     lines.append("Booth radix-2 multiplication: {} x {} ({}-bit operands)"
116                 .format(multiplicand, multiplier, bits))
117     lines.append("iter  Q0 Q-1  action    {:<{w}} {:<{w}} Q-1"
118                 .format("A register", "Q register", w=bits))
119     lines.append("-" * (33 + 2 * bits))
120     for step in steps:
121         lines.append(step.as_row(bits))
122     lines.append("-" * (33 + 2 * bits))
123     lines.append("product = {} (expected {}) iterations = {}".format(

```



```
124         .format(product, multiplicand * multiplier, len(steps)))
125     return "\n".join(lines)
```

src/division.py - restoring and non-restoring division

```
66 def restoring_divide(dividend: int, divisor: int,
67                       bits: int = 16) -> DivisionResult:
68     """Unsigned restoring division of a 2*bits dividend by a bits divisor.
69
70     Each iteration:
71         1. shift (A, Q) left one place
72         2. A = A - M
73         3. if A < 0 : restore it (A = A + M) and set Q0 = 0
74            else    : set Q0 = 1
75     """
76     if divisor == 0:
77         return DivisionResult((1 << bits) - 1, 0, [], divide_by_zero=True)
78
79     mask = (1 << bits) - 1
80     a, q = _split_dividend(dividend, bits)
81     m = divisor & mask
82     overflow = a >= m      # quotient would not fit in ``bits`` bits
83     steps: List[DivStep] = []
84
85     for i in range(bits):
86         # shift the (A, Q) pair left by one
87         a = ((a << 1) | (q >> (bits - 1))) & ((1 << (bits + 1)) - 1)
88         q = (q << 1) & mask
89
```

```

90     a_after_sub = a - m
91     if a_after_sub < 0:
92         action = "A=A-M <0 -> restore, q=0"
93         # A is restored, Q0 stays 0
94     else:
95         a = a_after_sub
96         q |= 1
97         action = "A=A-M >=0 -> keep, q=1"
98     steps.append(DivStep(i + 1, action, a, q))
99
100    return DivisionResult(q, a, steps, overflow=overflow)
101
102
103    def non_restoring_divide(dividend: int, divisor: int,
104                            bits: int = 16) -> DivisionResult:
105        """Unsigned non-restoring division of a 2*bits dividend by a bits divisor.
106
107        Each iteration performs exactly one add or subtract:
108            shift (A, Q) left
109            if A >= 0 : A = A - M else A = A + M
110            Q0 = 1 if the new A >= 0 else 0
111        A final correction step adds M back if the remainder ended up negative.
112        """
113        if divisor == 0:
114            return DivisionResult((1 << bits) - 1, 0, [], divide_by_zero=True)
115
116        mask = (1 << bits) - 1
117        a, q = _split_dividend(dividend, bits)

```

```

118     m = divisor & mask
119     overflow = a >= m
120     steps: List[DivStep] = []
121
122     for i in range(bits):
123         # arithmetic shift of the (A, Q) pair left by one
124         a = (a << 1) | (q >> (bits - 1))
125         q = (q << 1) & mask
126
127         if a >= 0:
128             a = a - m
129             action = "A>=0 -> A = A - M"
130         else:
131             a = a + m
132             action = "A< 0 -> A = A + M"
133
134         if a >= 0:
135             q |= 1
136         steps.append(DivStep(i + 1, action, a, q))
137
138     if a < 0:          # final restoration of the remainder
139         a = a + m
140     return DivisionResult(q, a, steps, overflow=overflow)
141
142
143 def signed_divide(dividend: int, divisor: int, bits: int = 16,
144                  method: str = "non_restoring") -> DivisionResult:
145     """Signed division built on top of an unsigned divider.

```

```

146
147 The magnitudes are divided and the signs are applied afterwards, which
148 gives truncation towards zero -- the same rule the Q8.8 DIVQ instruction
149 uses.
150 """
151 if divisor == 0:
152     return DivisionResult(0, 0, [], divide_by_zero=True)
153
154 negative_quotient = (dividend < 0) != (divisor < 0)
155 magnitude_dividend = abs(dividend)
156 magnitude_divisor = abs(divisor)
157
158 if method == "restoring":
159     result = restoring_divide(magnitude_dividend, magnitude_divisor, bits)
160 else:
161     result = non_restoring_divide(magnitude_dividend, magnitude_divisor, bits)
162
163 quotient = -result.quotient if negative_quotient else result.quotient
164 remainder = -result.remainder if dividend < 0 else result.remainder
165 return DivisionResult(quotient, remainder, result.steps,
166                       overflow=result.overflow)
167
168
169 def division_trace_table(dividend: int, divisor: int, bits: int = 8,
170                          method: str = "non_restoring") -> str:
171     """Human-readable trace of a division (used by the report)."""
172     if method == "restoring":
173         result = restoring_divide(dividend, divisor, bits)

```

```

174     else:
175         result = non_restoring_divide(dividend, divisor, bits)
176     lines = []
177     lines.append("{} division: {} / {} ({}-bit divisor)"
178                 .format(method.replace("_", "-"), dividend, divisor, bits))
179     lines.append("iter action          A Q")
180     lines.append("-" * (44 + bits))
181     for step in result.steps:
182         lines.append(step.as_row(bits))
183     lines.append("-" * (44 + bits))
184     lines.append("quotient = {} (expected {}), remainder = {} (expected {}), "
185                 "iterations = {}"
186                 .format(result.quotient, dividend // divisor,
187                         result.remainder, dividend % divisor,
188                         result.iterations))
189     return "\n".join(lines)

```

src/predictor.py - two-bit and static branch predictors

```

35 class TwoBitPredictor:
36     """Direct-mapped table of two-bit saturating counters."""
37
38     def __init__(self, entries: int = 16, initial_state: int = WEAKLY_NOT_TAKEN):
39         self.entries = entries
40         self.initial_state = initial_state
41         self.table: List[int] = [initial_state] * entries
42         self.lookups = 0
43         self.updates = 0
44         self.correct = 0

```

```

45     self.mispredicted = 0
46     #: per-branch-address statistics, filled in as the program runs
47     self.per_branch: Dict[int, Dict[str, int]] = {}
48
49     # -- table access -----
50     def index(self, pc: int) -> int:
51         return pc % self.entries
52
53     def state(self, pc: int) -> int:
54         return self.table[self.index(pc)]
55
56     def predict(self, pc: int) -> bool:
57         """Predict taken/not-taken for the branch at ``pc``."""
58         self.lookups += 1
59         return self.table[self.index(pc)] >= WEAKLY_TAKEN
60
61     def update(self, pc: int, taken: bool, predicted: bool) -> None:
62         """Train the counter once the branch outcome is known in EX."""
63         slot = self.index(pc)
64         self.updates += 1
65         if predicted == taken:
66             self.correct += 1
67         else:
68             self.mispredicted += 1
69
70         statistics = self.per_branch.setdefault(
71             pc, {"executed": 0, "taken": 0, "mispredicted": 0, "index": slot})
72         statistics["executed"] += 1

```

```

73     statistics["taken"] += 1 if taken else 0
74     statistics["mispredicted"] += 0 if predicted == taken else 1
75
76     if taken:
77         self.table[slot] = min(STRONGLY_TAKEN, self.table[slot] + 1)
78     else:
79         self.table[slot] = max(STRONGLY_NOT_TAKEN, self.table[slot] - 1)
80
81     # -- reporting -----
82     @property
83     def accuracy(self) -> float:
84         if self.updates == 0:
85             return 0.0
86         return self.correct / self.updates
87
88     def table_state(self) -> List[str]:
89         return [STATE_NAMES[value] for value in self.table]
90
91     def summary(self) -> Dict[str, object]:
92         return {
93             "entries": self.entries,
94             "lookups": self.lookups,
95             "updates": self.updates,
96             "correct": self.correct,
97             "mispredicted": self.mispredicted,
98             "accuracy": self.accuracy,
99             "final_table": self.table_state(),
100         }

```

```
101
102
103 class StaticNotTakenPredictor:
104     """Always predicts "not taken" -- the cheapest possible static scheme."""
105
106     def __init__(self, entries: int = 0):
107         self.entries = 0
108         self.lookups = 0
109         self.updates = 0
110         self.correct = 0
111         self.mispredicted = 0
112         self.per_branch: Dict[int, Dict[str, int]] = {}
113
114     def index(self, pc: int) -> int:
115         return 0
116
117     def predict(self, pc: int) -> bool:
118         self.lookups += 1
119         return False
120
121     def update(self, pc: int, taken: bool, predicted: bool) -> None:
122         self.updates += 1
123         if predicted == taken:
124             self.correct += 1
125         else:
126             self.mispredicted += 1
127         statistics = self.per_branch.setdefault(
128             pc, {"executed": 0, "taken": 0, "mispredicted": 0, "index": 0})
```



```

129     statistics["executed"] += 1
130     statistics["taken"] += 1 if taken else 0
131     statistics["mispredicted"] += 0 if predicted == taken else 1
132
133     @property
134     def accuracy(self) -> float:
135         if self.updates == 0:
136             return 0.0
137         return self.correct / self.updates
138
139     def table_state(self) -> List[str]:
140         return []
141
142     def summary(self) -> Dict[str, object]:
143         return {
144             "entries": 0,
145             "lookups": self.lookups,

```

src/pipeline.py - configuration/results data structures

```

111 class PipelineConfig:
112     name: str
113     overlap: bool = True
114     forwarding: bool = True
115     split_memory: bool = True
116     branch_policy: str = "two_bit"      # "stall" | "not_taken" | "two_bit"
117     predictor_entries: int = BRANCH_PREDICTOR_ENTRIES
118     description: str = ""
119

```

```

120 def feature_summary(self) -> Dict[str, str]:
121     return {
122         "pipelined": "yes" if self.overlap else "no (sequential)",
123         "forwarding": "yes" if self.forwarding else "no",
124         "memory": "split I/D" if self.split_memory else "unified",
125         "branch handling": {
126             "stall": "stall until EX",
127             "not_taken": "static not-taken",
128             "two_bit": "2-bit dynamic",
129         }[self.branch_policy],
130     }
131
132
133 @dataclass
134 class RunResult:
135     config: PipelineConfig
136     cycles: int = 0
137     instructions_retired: int = 0
138     instructions_fetched: int = 0
139     squashed: int = 0
140     stalls: Dict[str, int] = field(default_factory=dict)
141     stage_use: Dict[str, int] = field(default_factory=dict)
142     forwards: Dict[str, int] = field(default_factory=dict)
143     branch_stats: Dict[str, object] = field(default_factory=dict)
144     predictor_summary: Dict[str, object] = field(default_factory=dict)
145     registers: List[int] = field(default_factory=list)
146     data_memory: Dict[int, int] = field(default_factory=dict)
147     cycle_rows: List[Dict[str, str]] = field(default_factory=list)

```

```

148     timeline: Dict[int, Dict[str, object]] = field(default_factory=dict)
149     completion_latencies: List[int] = field(default_factory=list)
150     flags: Dict[str, int] = field(default_factory=dict)
151     completed: bool = False
152
153     @property
154     def total_stall_cycles(self) -> int:
155         return sum(self.stalls.values())
156
157     @property
158     def cpi(self) -> float:
159         if self.instructions_retired == 0:
160             return 0.0
161         return self.cycles / self.instructions_retired
162
163     @property
164     def average_latency_cycles(self) -> float:
165         if not self.completion_latencies:
166             return 0.0
167         return sum(self.completion_latencies) / len(self.completion_latencies)
168
169
170 # -----
171 # The processor
172 # -----
173
174 class Processor:

```

src/pipeline.py - forwarding and data-hazard detection

```
296 # -- forwarding -----
297 def _forward(self, register: int, latched_value: int,
298             ex_mem: Optional[EXMEM], mem_wb: Optional[MEMWB]) -> int:
299     if register == 0:
300         return 0
301     if not self.cfg.forwarding:
302         return latched_value
303     # EX/MEM has the newest value, but not for a LOAD: its data only
304     # arrives at the end of the MEM stage (that is the load-use hazard).
305     if (ex_mem is not None and ex_mem.writes_reg and ex_mem.rd == register
306         and not ex_mem.is_load):
307         self.forwards["EX_MEM"] += 1
308         return ex_mem.result
309     if mem_wb is not None and mem_wb.writes_reg and mem_wb.rd == register:
310         self.forwards["MEM_WB"] += 1
311         return mem_wb.value
312     return latched_value
313
314 # -- hazard detection -----
315 def _hazard(self, instruction: Instruction,
316            ex_mem: Optional[EXMEM]) -> Optional[str]:
317     """Return the stall reason for the instruction in ID, or None."""
318     sources = [r for r in instruction.source_registers() if r != 0]
319     if not sources:
320         return None
321
322     in_ex = self.ID_EX
```

```

323     if self.cfg.forwarding:
324         # With full forwarding only a load in EX still hurts.
325         if (in_ex is not None and in_ex.instr.is_load()
326             and in_ex.instr.rd != 0 and in_ex.instr.rd in sources):
327             return "data_load_use"
328         return None
329
330     # No forwarding: wait until the producer has written the register file.
331     if (in_ex is not None and in_ex.instr.writes_register()
332         and in_ex.instr.rd in sources):
333         return "data_load_use" if in_ex.instr.is_load() else "data_raw"
334     if (ex_mem is not None and ex_mem.writes_reg
335         and ex_mem.rd in sources):
336         return "data_load_use" if ex_mem.is_load else "data_raw"
337     return None

```

src/pipeline.py - branch resolution, cycle recording and run result

```

531     def _resolve_branch(self, execute: IDEX, outcome: dict,
532                         note: List[str]) -> bool:
533         """Compare the prediction with the real outcome, in the EX stage."""
534         taken = outcome["taken"]
535         target = outcome["target"]
536         self.branch_stats["executed"] += 1
537         self.branch_stats["taken" if taken else "not_taken"] += 1
538
539         if self.cfg.branch_policy == "stall":
540             self._pending_pc = target
541             self._release_branch_stall = True

```

```

542         note.append("branch resolved -> {}".format(target))
543         return False
544
545     self.predictor.update(execute.pc, taken, execute.predicted_taken)
546     predicted_target = (execute.predicted_target if execute.predicted_taken
547                         else execute.next_pc)
548     if predicted_target != target:
549         self.branch_stats["mispredicted"] += 1
550         self._pending_pc = target
551         note.append("branch MISPREDICTED (predicted {}, actual {})".format(
552             predicted_target, target))
553         return True
554     note.append("branch predicted correctly ({})" .format(target))
555     return False
556
557 def _datapath_busy(self) -> bool:
558     return any(latch is not None for latch in
559               (self.IF_ID, self.ID_EX, self.EX_MEM, self.MEM_WB))
560
561 @staticmethod
562 def _tag(latch) -> str:
563     """`12:LOAD` -- dynamic instruction number and mnemonic."""
564     if latch is None:
565         return "-"
566     return "{}:{}".format(latch.seq, latch.instr.short())
567
568 def _record_cycle(self, occupancy: Dict[str, str],
569                 note: List[str]) -> None:

```

```

570     if self.trace_cycles and self.cycle > self.trace_cycles:
571         return
572     row = {"cycle": self.cycle}
573     row.update(occupancy)
574     row["note"] = "; ".join(note)
575     self.cycle_rows.append(row)
576
577     # -- driver -----
578     def run(self, max_cycles: int = 2_000_000) -> RunResult:
579         while not self.finished and self.cycle < max_cycles:
580             self.step()
581             if (self.pc >= len(self.imem) and not self._datapath_busy()
582                 and self.ex_state is None):
583                 break      # ran off the end of instruction memory
584
585             predictor_summary = (self.predictor.summary() if self.predictor
586                                 else {"entries": 0, "lookups": 0, "updates": 0,
587                                     "correct": 0, "mispredicted": 0,
588                                     "accuracy": 0.0, "final_table": []})
589         return RunResult(
590             config=self.cfg,
591             cycles=self.cycle,
592             instructions_retired=self.instructions_retired,
593             instructions_fetched=self.instructions_fetched,
594             squashed=self.squashed,
595             stalls=dict(self.stalls),
596             stage_use=dict(self.stage_use),
597             forwards=dict(self.forwards),

```

```
598     branch_stats=dict(self.branch_stats),
599     predictor_summary=predictor_summary,
600     registers=list(self.regs),
601     data_memory=dict(self.dmem),
602     cycle_rows=list(self.cycle_rows),
603     timeline=dict(self.timeline),
604     completion_latencies=list(self.completion_latencies),
```

src/pipeline.py - six compared configurations

```
616     overlap=False, forwarding=False, split_memory=True, branch_policy="stall",
617     description="One instruction walks through all five stages before the "
618         "next one is fetched.")
619
620 BASIC_PIPELINE = PipelineConfig(
621     name="Basic pipeline",
622     overlap=True, forwarding=False, split_memory=False, branch_policy="stall",
623     description="Five stages overlapped, but no forwarding, one unified "
624         "memory port and fetch frozen on every branch.")
625
626 PIPELINE_FORWARDING = PipelineConfig(
627     name="Pipeline + forwarding",
628     overlap=True, forwarding=True, split_memory=False, branch_policy="stall",
629     description="Adds EX/MEM and MEM/WB forwarding paths.")
630
631 PIPELINE_SPLIT_MEMORY = PipelineConfig(
632     name="Pipeline + forwarding + split I/D memory",
633     overlap=True, forwarding=True, split_memory=True, branch_policy="stall",
634     description="Adds a Harvard memory so IF and MEM no longer fight over "
```



```

635         "one port.")
636
637 PIPELINE_STATIC_PREDICTION = PipelineConfig(
638     name="Pipeline + forwarding + split memory + static not-taken",
639     overlap=True, forwarding=True, split_memory=True, branch_policy="not_taken",
640     description="Adds the cheapest branch prediction: always guess "
641         "not-taken.")
642
643 OPTIMIZED_PIPELINE = PipelineConfig(
644     name="Optimized pipeline (2-bit predictor)",
645     overlap=True, forwarding=True, split_memory=True, branch_policy="two_bit",
646     description="Forwarding, Harvard memory and a two-bit dynamic branch "
647         "predictor.")
648

```

src/controller_program.py - irrigation controller instruction program

```

47 def build_controller_program() -> Program:
48     """Assemble the three-zone (or n-zone) irrigation controller."""
49     asm = Assembler("greenhouse_irrigation_controller")
50
51     asm.load(R_N, R_ZERO, cfg.ADDR_ZONE_COUNT, "r2 = number of zones")
52     asm.li(R_I, 0, "r1 = zone index")
53     asm.li(R_SENSOR, cfg.SENSOR_BASE, "r3 -> first sensor record")
54     asm.li(R_RESULT, cfg.RESULT_BASE, "r4 -> first result record")
55
56     # ----- loop
57     asm.label("ZONE_LOOP")
58     asm.slt(R_COND, R_I, R_N, "more zones to process?")

```

```

59  asm.beqz(R_COND, "DONE", "conditional branch: leave the loop")
60
61  asm.load(R_MOIST, R_SENSOR, cfg.SENSOR_OFF_MOISTURE, "soil moisture
62  %)
63  asm.load(R_TEMP, R_SENSOR, cfg.SENSOR_OFF_TEMP, "air temperature C")
64  asm.load(R_HUM, R_SENSOR, cfg.SENSOR_OFF_HUMIDITY, "relative humidity
65  %)
66
67  # ---- sensor range validation (invalid readings branch to FAULT) ----
68  asm.li(R_CONST, q(cfg.MOISTURE_MIN_VALID))
69  asm.slt(R_CMP, R_MOIST, R_CONST, "moisture below range?")
70  asm.bnez(R_CMP, "FAULT")
71  asm.li(R_CONST, q(cfg.MOISTURE_MAX_VALID))
72  asm.slt(R_CMP, R_CONST, R_MOIST, "moisture above range?")
73  asm.bnez(R_CMP, "FAULT")
74  asm.li(R_CONST, q(cfg.TEMP_MIN_VALID))
75  asm.slt(R_CMP, R_TEMP, R_CONST, "temperature below range?")
76  asm.bnez(R_CMP, "FAULT")
77  asm.li(R_CONST, q(cfg.TEMP_MAX_VALID))
78  asm.slt(R_CMP, R_CONST, R_TEMP, "temperature above range?")
79  asm.bnez(R_CMP, "FAULT")
80  asm.li(R_CONST, q(cfg.HUMIDITY_MIN_VALID))
81  asm.slt(R_CMP, R_HUM, R_CONST, "humidity below range?")
82  asm.bnez(R_CMP, "FAULT")
83  asm.li(R_CONST, q(cfg.HUMIDITY_MAX_VALID))
84  asm.slt(R_CMP, R_CONST, R_HUM, "humidity above range?")
85  asm.bnez(R_CMP, "FAULT")

```

```

85  # ---- moisture deficit ----
86  asm.li(R_CONST, q(cfg.MOISTURE_TARGET))
87  asm.sub(R_DEFICIT, R_CONST, R_MOIST, "deficit = target - moisture")
88  asm.li(R_CONST, 0)
89  asm.slt(R_CMP, R_CONST, R_DEFICIT, "deficit > 0 ?")
90  asm.beqz(R_CMP, "SOIL_WET", "irrigation branch: soil already wet")
91
92  # ---- evapotranspiration factor ----
93  asm.li(R_CONST, q(cfg.TEMP_REFERENCE))
94  asm.sub(R_TTERM, R_TEMP, R_CONST, "temp - 25 C")
95  asm.li(R_CONST, q(cfg.K_TEMP))
96  asm.mulq(R_TTERM, R_TTERM, R_CONST, "Booth multiply: k_temp x dT")
97  asm.li(R_CONST, q(cfg.HUMIDITY_REFERENCE))
98  asm.sub(R_HTERM, R_CONST, R_HUM, "60 %RH - humidity")
99  asm.li(R_CONST, q(cfg.K_HUMIDITY))
100  asm.mulq(R_HTERM, R_HTERM, R_CONST, "Booth multiply: k_hum x dRH")
101  asm.li(R_ET, q(1.0))
102  asm.add(R_ET, R_ET, R_TTERM)
103  asm.add(R_ET, R_ET, R_HTERM, "ET = 1 + temp term + humidity term")
104
105  asm.li(R_CONST, q(cfg.ET_MIN))
106  asm.slt(R_CMP, R_ET, R_CONST, "ET below the lower clamp?")
107  asm.beqz(R_CMP, "ET_HIGH_CLAMP")
108  asm.mov(R_ET, R_CONST, "clamp ET to the minimum")
109  asm.label("ET_HIGH_CLAMP")
110  asm.li(R_CONST, q(cfg.ET_MAX))
111  asm.slt(R_CMP, R_CONST, R_ET, "ET above the upper clamp?")
112  asm.beqz(R_CMP, "ET_READY")

```

```

113  asm.mov(R_ET, R_CONST, "clamp ET to the maximum")
114  asm.label("ET_READY")
115
116  # ---- watering time and volume ----
117  asm.li(R_CONST, q(cfg.MINUTES_PER_PERCENT))
118  asm.mulq(R_MINUTES, R_DEFICIT, R_CONST, "Booth multiply: base minutes")
119  asm.mulq(R_MINUTES, R_MINUTES, R_ET, "Booth multiply: apply ET factor")
120  asm.li(R_CONST, q(cfg.MAX_MINUTES))
121  asm.slt(R_CMP, R_CONST, R_MINUTES, "longer than the valve limit?")
122  asm.beqz(R_CMP, "MINUTES_READY")
123  asm.mov(R_MINUTES, R_CONST, "clamp to the maximum valve time")
124  asm.label("MINUTES_READY")
125  asm.li(R_CONST, q(cfg.FLOW_L_PER_MIN))
126  asm.mulq(R_LITRES, R_MINUTES, R_CONST, "Booth multiply: litres needed")
127
128  # ---- solar battery check ----
129  asm.load(R_BATTERY, R_ZERO, cfg.ADDR_BATTERY_PERCENT, "solar
battery %")
130  asm.li(R_CONST, q(cfg.BATTERY_MIN_PERCENT))
131  asm.slt(R_CMP, R_BATTERY, R_CONST, "battery too low to run the pump?")
132  asm.bnez(R_CMP, "DEFER_BATTERY", "irrigation branch: defer on low solar")
133
134  # ---- rainwater tank check ----
135  asm.load(R_TANK, R_ZERO, cfg.ADDR_TANK_LITRES, "rainwater tank litres")
136  asm.li(R_CONST, q(cfg.TANK_RESERVE_L))
137  asm.sub(R_AVAIL, R_TANK, R_CONST, "usable = tank - reserve")
138  asm.li(R_CONST, 0)
139  asm.slt(R_CMP, R_CONST, R_AVAIL, "any usable rainwater left?")

```

```
140  asm.beqz(R_CMP, "DEFER_WATER", "irrigation branch: tank empty")
141  asm.slt(R_CMP, R_AVAIL, R_LITRES, "tank cannot supply the full dose?")
142  asm.beqz(R_CMP, "IRRIGATE_FULL")
143
144  # ---- reduced dose ----
145  asm.mov(R_LITRES, R_AVAIL, "give whatever the tank still holds")
146  asm.li(R_CONST, q(cfg.FLOW_L_PER_MIN))
147  asm.divq(R_MINUTES, R_LITRES, R_CONST, "non-restoring divide: new
minutes")
148  asm.li(R_DECISION, cfg.DECISION_REDUCED_TANK_LIMIT)
149  asm.jmp("COMMIT")
150
151  asm.label("IRRIGATE_FULL")
152  asm.li(R_CONST, q(cfg.FLOW_L_PER_MIN))
153  asm.divq(R_MINUTES, R_LITRES, R_CONST, "non-restoring divide: valve time")
154  asm.li(R_DECISION, cfg.DECISION_IRRIGATE_FULL)
155  asm.jmp("COMMIT")
156
157  asm.label("DEFER_BATTERY")
158  asm.li(R_DECISION, cfg.DECISION_DEFER_LOW_BATTERY)
159  asm.li(R_MINUTES, 0)
160  asm.li(R_LITRES, 0)
161  asm.jmp("COMMIT")
162
163  asm.label("DEFER_WATER")
164  asm.li(R_DECISION, cfg.DECISION_DEFER_NO_WATER)
165  asm.li(R_MINUTES, 0)
166  asm.li(R_LITRES, 0)
```

```
167  asm.jump("COMMIT")
168
169  asm.label("SOIL_WET")
170  asm.li(R_DECISION, cfg.DECISION_SOIL_WET)
171  asm.li(R_MINUTES, 0)
172  asm.li(R_LITRES, 0)
173  asm.jump("COMMIT")
174
175  asm.label("FAULT")
176  asm.li(R_DECISION, cfg.DECISION_SENSOR_FAULT)
177  asm.li(R_MINUTES, 0)
178  asm.li(R_LITRES, 0)
179  asm.li(R_DEFICIT, 0, "no trustworthy deficit for a faulty sensor")
180
181  # ---- write the results back ----
182  asm.label("COMMIT")
183  asm.li(R_CONST, q(cfg.MOISTURE_TARGET))
184  asm.divq(R_RATIO, R_DEFICIT, R_CONST, "non-restoring divide: deficit ratio")
185  asm.load(R_TANK, R_ZERO, cfg.ADDR_TANK_LITRES)
186  asm.sub(R_TANK, R_TANK, R_LITRES, "load-use hazard: take water from tank")
187  asm.store(R_TANK, R_ZERO, cfg.ADDR_TANK_LITRES)
188  asm.load(R_TOTAL, R_ZERO, cfg.ADDR_TOTAL_LITRES)
189  asm.add(R_TOTAL, R_TOTAL, R_LITRES, "load-use hazard: running water total")
190  asm.store(R_TOTAL, R_ZERO, cfg.ADDR_TOTAL_LITRES)
191
192  asm.store(R_DECISION, R_RESULT, cfg.RESULT_OFF_DECISION)
193  asm.store(R_MINUTES, R_RESULT, cfg.RESULT_OFF_MINUTES)
194  asm.store(R_LITRES, R_RESULT, cfg.RESULT_OFF_LITRES)
```

```

195     asm.store(R_RATIO, R_RESULT, cfg.RESULT_OFF_DEFICIT_RATIO)
196
197     asm.addi(R_I, R_I, 1, "next zone")
198     asm.addi(R_SENSOR, R_SENSOR, cfg.SENSOR_STRIDE)
199     asm.addi(R_RESULT, R_RESULT, cfg.RESULT_STRIDE)
200     asm.jump("ZONE_LOOP", "loop back-edge: the predictor learns this one")
201
202     asm.label("DONE")
203     asm.halt()
204     return asm.assemble()

```

web/api.py - dashboard simulation endpoint (calls the real simulator)

```

160 def simulate(request: Optional[Dict[str, Any]] = None) -> Dict[str, Any]:
161     """Run the real controller program on the zones the browser sent."""
162     request = request or {}
163     zones_in = request.get("zones")
164     if not zones_in:
165         zones_in = default_inputs()["zones"]
166     if len(zones_in) > MAX_ZONES:
167         raise ValueError("at most {} zones can be simulated at
168         once".format(MAX_ZONES))
169
170     fallback = sensors.load_scenarios()[0]
171     tank = _number(request.get("tank_litres"), fallback["tank_litres"],
172                    "tank_litres")
173     battery = _number(request.get("battery_percent"), fallback["battery_percent"],
174                       "battery_percent")
175     view_cycles = int(request.get("view_cycles") or DEFAULT_VIEW_CYCLES)

```

```

175 view_instructions = int(request.get("view_instructions")
176                          or DEFAULT_VIEW_INSTRUCTIONS)
177
178 # ---- 1. condition the readings with the project's own front end -----
179 conditioned = []
180 for index, zone in enumerate(zones_in):
181     case = {
182         "id": index + 1,
183         "name": (zone.get("name") or "zone_{}".format(index)),
184         "category": zone.get("category", "web input"),
185         "moisture_percent": _sensor_value(zone.get("moisture")),
186         "temperature_c": _sensor_value(zone.get("temperature")),
187         "humidity_percent": _sensor_value(zone.get("humidity")),
188         "expect_sensor_fault": False,
189         "notes": zone.get("notes", ""),
190     }
191     conditioned.append(sensors.condition_case(case))
192
193 words = sensors.sensor_words(conditioned)
194 zone_count = len(words)
195
196 # ---- 2. execute the program on every processor configuration -----
197 program = build_controller_program()
198 memory = build_data_memory(words, tank, battery)
199
200 runs = []
201 for config in ABLATION_CONFIGS:
202     processor = Processor(program, memory, config,

```



```

203         trace_cycles=INTERNAL_TRACE_CYCLES,
204         trace_instructions=INTERNAL_TRACE_INSTRUCTIONS)
205     runs.append(processor.run())
206
207     # ---- 3. cross-checks -----
208     decisions = read_results(runs[0].data_memory, zone_count)
209     identical = all(read_results(run.data_memory, zone_count) == decisions
210                     for run in runs[1:])
211     reference = run_reference(words, tank, battery)
212     compared_fields = ("decision", "minutes_raw", "litres_raw", "deficit_ratio_raw")
213     matches_reference = all(
214         all(hardware[field] == model[field] for field in compared_fields)
215         for hardware, model in zip(decisions, reference["zones"]))
216
217     # ---- 4. metrics -----
218     comparison = metrics.compare(runs, zone_count)
219     performance_rows = [_performance_row(entry) for entry in comparison]
220     baseline_water = baseline_fixed_schedule(zone_count)
221     total_litres = from_q88(runs[0].data_memory.get(cfg.ADDR_TOTAL_LITRES, 0))
222     tank_left = from_q88(runs[0].data_memory.get(cfg.ADDR_TANK_LITRES, 0))
223
224     return {
225         "inputs": {
226             "zones": [_conditioned_summary(entry) for entry in conditioned],
227             "tank_litres": tank,
228             "battery_percent": battery,
229             "zone_count": zone_count,
230         },

```

```

231     "program": _program_summary(program),
232     "runs": [_run_summary(run, view_cycles, view_instructions) for run in runs],
233     "performance": performance_rows,
234     "headline": [row for row in performance_rows
235                   if row["configuration"] in (SEQUENTIAL.name,
236                                               BASIC_PIPELINE.name,
237                                               OPTIMIZED_PIPELINE.name)],
238     "baseline_vs_optimized": _baseline_vs_optimized(performance_rows),
239     "agriculture": {
240         "decisions": [_decision_summary(entry, decision)
241                       for entry, decision in zip(conditioned, decisions)],
242         "total_litres": total_litres,
243         "tank_litres_left": tank_left,
244         "water": metrics.water_summary(total_litres,
245                                       baseline_water["total_litres"],
246                                       zone_count),
247         "fixed_schedule_baseline": baseline_water,
248     },
249     "verification": {
250         "identical_across_configurations": identical,
251         "matches_reference_model": matches_reference,
252         "configurations_compared": len(runs),
253         "all_runs_completed": all(run.completed for run in runs),
254     },
255     "ideal_speedup": metrics.ideal_speedup(),
256     "view": {"cycles": view_cycles, "instructions": view_instructions},
257 }
258

```

Appendix B - Submission File Structure

Path	Purpose
src/	Computer-architecture simulator, arithmetic modules, ISA, controller and reference model
tests/	Processor, arithmetic, sensor, end-to-end and web/API automated tests
data/	13 controlled sensor test cases and 3 shared-resource scenarios
results/	Generated CSV/JSON/TXT performance, hazards, cycles, decisions and test output
web/	HTML/CSS/JavaScript dashboard and Python API/server integration
docs/ screenshots/ web_demo/	Executed dashboard screenshots
run_tests.py	Automated test runner
run_web.py	Local dashboard launcher